Coinbase Mesh API & Overledger Integration Analysis

This document provides a comprehensive analysis of integrating Coinbase Mesh (formerly Rosetta) with the Overledger platform. It outlines the

mapping feasibility, technical requirements, key challenges, and strategic considerations.

Key Insights

General Feasibility: *Mapping Overledger's read/write functions to the Coinbase Mesh API is possible. The Prep-Sign-Execute flow aligns*

well with Mesh's Construction API.

Significant Complexity: *The mapping process is not straightforward, especially for smart contract interactions. Mesh's endpoints for reading*

(/call) and preparing (/payloads) smart contract transactions are vague and require further technical investigation.

Operational Hurdle: *The official documentation mandates running a dedicated local blockchain node for each Mesh implementation within the*

same Docker container. This poses a significant resource, cost, and operational overhead.

Documentation Discrepancy: *A critical contradiction exists between official documentation and the reference implementation for Ethereum.*

The reference code appears to allow connecting to an external node, which, if true, could mitigate the primary operational hurdle. This

ambiguity is a major risk factor.

Limited Ecosystem: *The Mesh standard appears to have low adoption, with limited community resources, tutorials, and support outside of*

Coinbase's official repositories. Most reference materials are written in Golang.

Overledger to Mesh API Mapping

The analysis reveals a variable level of compatibility between Overledger commands and Mesh API endpoints. While some functions map directly,

others present notable gaps.

Read Operations: Overledger Read → Mesh Data API

Overledger Coinbase Mesh Method Endpoint

Analysis & Gaps

Account Balance

Block Data

Get balance by address

Get block by ID/hash

/account/balance

/block

Good Alignment. Mesh offers enhanced functionality, such as retrieving balances for specific tokens by providing a token contract address.

Good Alignment. Both systems use a block identifier to retrieve block information.

Transaction Data

Get **Misalignment.** Overledger can retrieve a transaction using only transaction /block/transactionits ID. Mesh **requires both the Block ID and the Transaction** by ID/hash **ID**, creating a dependency and potential friction.

Read Smart Contract

Read a smart contract function

/call

High Complexity. Mesh's /call endpoint is a generic method for network-specific procedure calls. Mapping Overledger's structured read calls will require significant investigation to understand the required request structure.

> ***Write Operations: Overledger Write → Mesh Construction API***
>
> *The standard Overledger transaction lifecycle (Prepare → Sign → Execute) maps conceptually well to Mesh's payloads → parse → submit flow.*
>
> ***Overledger Coinbase Mesh Method Endpoint***

Analysis & Gaps

Prepare Transaction

> ***High Complexity for Smart Contracts.*** *While straightforward for simple transfers, constructing the*

prepare /construction/payloadsoperations object for a smart contract call is not clearly documented. Further investigation is needed to determine the correct payload structure.

Sign Transaction

Execute Transaction

sign

execute

/construction/parse

/construction/submit

N/A. Comparison is omitted, as signing will be handled by a proprietary solution (Overwallet).

Good Alignment. Both endpoints require the signed transaction payload for broadcast.

Implementation & Technical Considerations

Standard Implementation Process

A compliant Mesh implementation requires the following steps:

*1. **Adhere to Specifications:** Ensure all API request/response formats match the official API Reference (https://docs.cloud.coinbase.com/mesh/reference/api-reference)*

(https://docs.cloud.coinbase.com/mesh/reference/api-reference).
(https://docs.cloud.coinbase.com/mesh/reference/api-reference)

*2. **Implement Endpoints:** Develop all required endpoints as defined in the Mesh documentation.*

*3. **Deploy with Node:** Package the Mesh implementation and a blockchain node together using Docker.*

*4. **Test for Compliance:** Validate the implementation using the mesh-cli tool to check against specifications.*

Support for Specific Blockchain Features

***Memos/Destination Tags:** Supported for chains like XRPL via the memo field within the metadata or options objects in API calls.*

***Non-Balance-Changing Operations:** While not mandatory, it is strongly recommended to include non-financial operations (e.g., governance*

votes, validator nominations) in the /transaction endpoint to provide a complete view of on-chain activity.

Data Model for New Implementations

For customers providing their own Mesh implementation, the process is straightforward:

The customer is responsible for building, maintaining, and running the implementation.

Integration with Overledger is achieved by providing the hosted API endpoint URL.

Identified Challenges & Strategic Risks

Challenge Description Impact

Local Node Requirement

Official documentation mandates running **High.** Discourages adoption due to high operational a blockchain node locally within the same overhead. However, the mesh-ethereum repository Docker container as the Mesh service. suggests an GethEnv variable may permit external This is resource-intensive and costly. node connections, creating critical uncertainty.

Key processes, particularly for preparing **Documentation** smart contract transactions via

& Clarity /construction/payloads, are poorly documented and ambiguous.

High. Creates significant implementation risk and requires direct clarification from Coinbase, delaying development and increasing effort.

Ecosystem & Community

The Mesh standard has not achieved widespread popularity. Resources are scarce, and reference implementations are almost exclusively in Golang.

Medium. Limits community support and may create a knowledge dependency on Go. Lack of diverse examples makes troubleshooting difficult.

Lack of NFT Support

NFTs are not mentioned in the official **Medium.** Represents a significant feature gap for documentation or API reference. Support modern blockchain applications. Any

might be possible via generic metadata implementation would be a custom, non-standard fields, but this is unconfirmed. extension.

Action Guidelines & Strategic Recommendations

*1. **Prioritize Clarification from Coinbase:** Before committing to a full-scale integration, it is essential to resolve the most critical ambiguities. The*

primary questions for Coinbase are:

***External Node Connectivity:** Is connecting to an external or managed node (e.g., Infura, Alchemy) officially supported, despite*

documentation stating otherwise?

***Smart Contract Preparation:** Request detailed examples or documentation for constructing a /construction/payloads request*

for a typical smart contract method call.

***NFT Support:** Inquire about the official recommendation for handling ERC-721/ERC-1155 tokens.*

*2. **Evaluate a Phased Approach:** If proceeding, consider a partial implementation focused on well-aligned features first (e.g., balance and*

transfer operations) while continuing to investigate the more complex smart contract functionalities.

*3. **Consider the Strategic Alternative:** The analysis highlights significant friction in adopting Mesh. A viable alternative is to **open-source***

***select Overledger components**. This strategy could offer greater flexibility, reduce integration complexity for developers, and foster an*

ecosystem around a proprietary standard without the limitations imposed by Mesh.

Coinbase Mesh: A Comprehensive Technical Overview

Executive Summary

Coinbase Mesh is an open-source, blockchain-agnostic specification and toolset designed to standardize interactions with Distributed Ledger

Technologies (DLTs). Its primary strategic purpose is to create a universal API layer that simplifies, accelerates, and improves the reliability of

blockchain integration for developers. By abstracting the unique complexities of different blockchains, Mesh allows applications to be built on any

supported DLT using a single, consistent interface.

This standard appears to be an evolution or rebranding of the earlier "Rosetta" API, sharing many of its core principles. It is crucial to distinguish the

***Coinbase Mesh specification** from unaffiliated, similarly named DeFi projects like "Mesh Protocol" (on Solana) and "Meshswap Protocol" (on*

Polygon), or third-party services like "Mesh Connect" that use Coinbase's consumer APIs. This document focuses exclusively on the open-source

technical standard provided by Coinbase for DLT integration.

Core Architecture

The Mesh architecture is designed as a middleware layer that sits between a client application and a blockchain node. Its modular design decouples

application logic from chain-specific protocols.

Key Architectural Components:

Component

Mesh API Specification

Description

An **OpenAPI 3.0 specification** that defines a universal set of RESTful endpoints, data models, and communication patterns for all blockchain interactions. This is the "contract" for the standard.

A service that implements the Mesh API for a *specific* blockchain. It acts as a translator, **Implementation** converting the standardized Mesh API calls into the native RPC calls of the target blockchain

Blockchain Node

Developer Tooling

Deployment Model

A full node of the target DLT (e.g., Geth for Ethereum, Bitcoin Core for Bitcoin). The Mesh implementation communicates directly with this node to read chain data and broadcast transactions.

A suite of tools to support development:

- mesh-cli: A command-line tool to test and validate a Mesh implementation's compliance with the specification.

- mesh-sdk-go: An SDK to accelerate the development of Mesh implementations in Golang.

The standard deployment model involves packaging the **Mesh Implementation** and the corresponding **Blockchain Node** together in a single Docker container to ensure tight coupling and operational consistency.

> *An implementation's internal logic typically consists of several packages from the mesh-sdk-go:*
>
> ***Syncer:*** *Fetches and processes blocks from the node.*
>
> ***Parser:*** *Interprets block data into the standardized Mesh models.*
>
> ***Storage:*** *Persists indexed data, often using memory-mapped files and compression (e.g., Zstandard) for efficiency.*
>
> ***Server:*** *Exposes the Mesh API endpoints.*
>
> ***Data Models & Specifications***

The source of truth for all Mesh data models and API definitions is the coinbase/mesh-specifications GitHub repository. The specifications are

*written in the **OpenAPI 3.0** format, which allows for automatic generation of client and server code.*

Core Data Model Principles:

Standardization: *Common blockchain concepts like Block, Transaction, AccountIdentifier, and Amount are standardized into*

universal models.

Extensibility: *The metadata field is available in many objects to accommodate chain-specific information without breaking the core standard.*

Clarity: *Enumerated types are used for critical fields, such as CurveType (cryptographic curves like secp256k1) and SignatureType (e.g.,*

ecdsa, ecdsa_recovery), ensuring cross-chain compatibility.

By defining these primitives in a single specification, Mesh ensures that developers can interact with vastly different blockchains using the

same objects and logic.

API Specification

*The Mesh API is divided into two primary categories: the **Data API** (for reading blockchain state) and the **Construction API** (for creating and*

broadcasting transactions).

Data API (Read Operations)

Endpoint

*/network/**

Functionality

Get information about the network status, options, and peers.

Key Considerations

Essential for client applications to discover chain features.

/account/balance Retrieve the balance of a Can specify tokens/sub-accounts for more granular queries.

Get the contents of a

/block block by its hash or The primary method for retrieving block data. index (height).

Get a specific /block/transactiontransaction within a

block.

Misalignment Risk: Requires both block_identifier and transaction_identifier, which can be less efficient than querying by transaction hash alone.

/mempool

/call

Get transactions currently in the mempool.

Perform a read-only call to a smart contract.

Implementation is optional for chains without robust mempool monitoring.

High Complexity: The request/response format is generic and requires detailed knowledge of the target chain's calling conventions. Documentation is sparse.

Construction API (Write Operations)

This API follows a Prepare -> Sign -> Execute flow, which safely decouples transaction creation from key management.

Endpoint Functionality Key Considerations

Endpoint Functionality Key Considerations

Create an

/construction/payloadstransaction payload.

High Complexity for Smart Contracts: The structure of the operations object for smart contract interactions is poorly documented and a significant implementation hurdle.

/construction/parse

/construction/combine

/construction/submit

Parse a

transaction to Used to verify transaction details before signing. verify its contents.

Combine an

unsigned payload Creates the final, signed transaction object. with signatures.

Broadcast a

signed transaction The final step to execute a transaction. to the network.

Supported DLTs and Protocols

DLT Support *Mesh is designed to be* ***blockchain-agnostic****. Support for a specific DLT is determined by the availability of a compliant Mesh*

implementation.

Official/Reference Implementations: *Coinbase provides reference implementations for* ***Bitcoin*** *(mesh-bitcoin) and* ***Ethereum*** *(mesh-*

ethereum).

Community Implementations: *The mesh-ecosystem repository serves as a registry for community-contributed implementations for other*

DLTs.

Communication Protocols

Application-to-Mesh: *The Mesh API uses standard* ***REST over HTTP(S)****.*

Mesh-to-Node: *The communication protocol between the Mesh implementation and the blockchain node is specific to the DLT (e.g.,* ***JSON-***

RPC *for Ethereum).*

Integration & Implementation Guide

Developers can interact with Mesh in two ways: integrating an application with an existing implementation or creating a new implementation for an

unsupported blockchain.

Implementation Steps for a New Blockchain:

1. ***Analyze the Blockchain:*** *Thoroughly understand the blockchain's transaction structure, account model, and RPC interface.*

2. ***Reference the Specification:*** *Use the mesh-specifications repository as the definitive guide for required endpoints and data models.*

3. ***Develop the API Endpoints:*** *Implement the Data and Construction APIs. The mesh-sdk-go is recommended for projects using Golang.*

4. ***Package and Deploy:*** *Create a Dockerfile to package the implementation alongside its corresponding blockchain node.*

*5. **Test for Compliance:** Use the mesh-cli tool extensively to validate that the implementation adheres to the Mesh standard.*

 # Example: Using mesh-cli to check the implementation

mesh-cli check:data --configuration-file <path-to-config>.json

mesh-cli check:construction --configuration-file <path-to-config>.json

Initial Overledger Integration Feasibility

An analysis of integrating Quant's Overledger platform with the Coinbase Mesh standard reveals the following:

***General Feasibility: Possible.** The core read/write functions of Overledger can be mapped to Mesh's Data and Construction APIs. The*

Prep-Sign-Execute transaction lifecycle aligns conceptually well with Mesh's /construction/ endpoints.*

Key Challenges & Risks:

*1. **Smart Contract Complexity:** Mapping smart contract reads (/call) and transaction preparation (/construction/payloads) is*

highly complex due to vague documentation and the generic nature of the endpoints. This represents the single largest technical risk.

*2. **Operational Overhead:** The official mandate to run a dedicated blockchain node locally within the same Docker container presents*

significant resource, cost, and maintenance overhead, which may deter adoption.

*3. **Documentation Ambiguity:** A critical contradiction exists between the official documentation (mandating a local node) and the*

reference implementation for Ethereum, which appears to allow connecting to an external node via an environment variable

*(GethEnv). **Clarifying this is essential.***

*4. **Limited Ecosystem:** The standard has not achieved widespread adoption. Community support, tutorials, and diverse reference*

implementations are scarce, with most existing resources written in Golang.

*5. **NFT Support:** Non-Fungible Tokens (NFTs) are not explicitly mentioned in the standard. Supporting them would likely require custom,*

non-standard use of the metadata field.

Strategic Recommendation:

> ***Prioritize Clarification:*** *Engage directly with Coinbase support or the Mesh developer community to resolve the critical ambiguities*
>
> *around* ***external node connectivity*** *and* ***smart contract payload construction****.*
>
> ***Adopt a Phased Approach:*** *If proceeding, begin with a partial implementation focusing on well-aligned features (e.g., token transfers)*
>
> *while investigating the more complex functionalities.*
>
> ***Evaluate Alternatives:*** *Given the significant friction, consider the strategic alternative of open-sourcing select Overledger components*
>
> *to build a proprietary, more flexible standard without the limitations imposed by Mesh.*
>
> ***Comparative Analysis Report: Quant Overledger vs. Coinbase Mesh***

This report provides a rigorous comparative analysis of the Quant Overledger and Coinbase Mesh platforms. The analysis is based on their respective

technical documentation, evaluating their core architecture, data models, API design, transaction handling, and security. The final synthesis assesses

the feasibility of creating a unified connector, considering the context of emerging ISO DLT standards.

1. Executive Summary: Two Philosophies of Interoperability

Quant Overledger and Coinbase Mesh represent two fundamentally different approaches to achieving blockchain interoperability.

***Quant Overledger** operates as a centralized **API Gateway as-a-Service**. It provides a fully managed, proprietary abstraction layer that offers*

developers a single, unified REST API to interact with multiple DLTs without needing to run any blockchain infrastructure. Its core value

proposition is simplicity, speed, and managed service.

***Coinbase Mesh** is an open-source **Standardized Node Interface Specification**. It is not a service but a blueprint (an OpenAPI specification)*

for how a blockchain node should expose its functions. Implementers must run a Mesh-compliant service alongside a full blockchain node,

effectively creating a standardized wrapper. Its core value proposition is decentralization, control, and open standards.

This fundamental difference in philosophy—Gateway vs. Standard—drives all subsequent architectural, operational, and security distinctions.

Feature Core Model

Infrastructure

Primary Goal

Transaction Flow

Authentication

Ecosystem

Quant Overledger API Gateway (SaaS)

Fully managed by Quant

Simplify developer access to many DLTs

2-Step: Prepare -> Execute

Centralized OAuth 2.0

Proprietary, closed-source

Coinbase Mesh

API Specification (Self-hosted)

Self-managed by the implementer (Node + Wrapper)

Standardize how any DLT node communicates

4-Step: Payloads -> Parse -> Combine -> Submit

Implementation-specific (e.g., API Key, mTLS)

Open-source, community-driven (but limited)

> ***2. Data Models & Abstractions***
>
> *Both platforms standardize DLT concepts into unified models, but their scope and implementation differ.*

Concept Quant Overledger Coinbase Mesh

Target DLT

A location object (technology, A Mesh implementation is *specific to one chain*. The server to specify the target chain. endpoint itself defines the target DLT.

Accounts

Identified by an accountId string within origin or destination arrays.

A structured AccountIdentifier object with address and optional sub_account fields.

> *A single, high-level JSON model with Multiple granular models: Transaction, Operation, destination. SigningPayload. Operations are balance-changing actions.*

Blocks

Abstracted block data, searchable by A standardized Block object containing an array of blockId (hash or number). Transaction objects.

Smart Contracts

Modeled via smartContract objects within requests, specifying address, function, and inputs.

No first-class smart contract model. Interactions are built generically using the Operation model and metadata, which is a noted area of high complexity.

Events are handled via Webhooks. **Events/NFTs** NFTs are not explicitly modeled in

> *the core documentation.*

Events are not explicitly modeled. NFTs are not mentioned and would require custom use of the metadata field.

Key Insight: *Overledger's abstractions are higher-level and designed for application developers. Mesh's abstractions are lower-level, designed for*

node implementers to map raw chain data to a standard format. Mesh's use of metadata for extensibility offers flexibility but risks creating

implementation-specific dialects, undermining the goal of a universal standard.

3. API Structure & Philosophy

Both platforms use RESTful APIs, but their structure reflects their underlying architectural philosophy.

Quant Overledger: Action-Oriented API

Overledger's API is structured around developer actions: preparing transactions, executing them, and searching for data.

Authentication: *Centralized Bearer <JWT> token via OAuth 2.0 (client_credentials grant type). Simple and standard for SaaS*

platforms.

Endpoints: *Grouped by function (/preparation, /execution, /search, /webhook). This is intuitive for an application developer who*

thinks in terms of tasks.

Developer Experience: *High. A single API key provides access to all supported DLTs. The documentation is geared toward building*

applications quickly. The two-step transaction flow is a clear, easy-to-follow pattern.

Coinbase Mesh: Data-Oriented API

Mesh's API is divided into reading data vs. constructing transactions, reflecting a focus on the node's capabilities.

Architecture:

Authentication: *Not part of the specification. The implementer is responsible for securing their endpoint. This offers flexibility but lacks a*

universal standard for access control.

Endpoints:

Data API (/network, /block, /account): *For all read operations.*

Construction API (/construction/*): *For all write (transaction-building) operations.*

This separation is logical from an infrastructure perspective (separating read replicas from write nodes) but requires the developer to

understand the more granular steps of transaction creation.

Developer Experience: *Lower, with a steeper learning curve. The developer must first find or host a Mesh implementation for their target DLT.*

The documentation is aimed at implementers, and crucial areas like smart contract interaction are noted to be vague.

4. Transaction Lifecycle

The transaction lifecycle is a critical point of divergence, highlighting the different security and control models.

Overledger's 2-Step "Prepare & Execute" Flow

1. ***POST /v2/preparation/transaction:***

Client sends: *High-level transaction intent (e.g., "send 1 ETH from A to B").*

Overledger returns: *A DLT-native, unsigned transaction payload.*

Purpose: Offloads the complexity of transaction formatting to the Overledger platform.

2. ***POST /v2/execution/transaction:***

Client sends: *The signed payload from step 1.*

Overledger returns: *A confirmation that the transaction has been broadcast.*

Purpose: Ensures private keys never touch Overledger's servers. Overledger acts as a secure broadcast channel.

This model is simple and highly secure from the user's perspective.

Mesh's 4-Step "Construct, Sign & Submit" Flow

1. ***POST /construction/payloads:***

Client sends: *A list of intended operations (e.g., balance changes).*

Mesh Server returns: *An unsigned transaction payload and the data that needs to be signed.*

Purpose: Create the unsigned transaction.

2. ***POST /construction/parse (Optional but Recommended):***

Client sends: *The unsigned transaction from step 1.*

Mesh Server returns: *A human-readable list of operations.*

Purpose: Allows the signer (e.g., a wallet) to verify the transaction's content before signing.

3. ***POST /construction/combine:***

Client sends: *The unsigned payload and the generated signature.*

Mesh Server returns: *A final, signed, network-ready transaction.*

Purpose: Assemble the complete, signed transaction.

4. ***POST /construction/submit:***

Client sends: *The signed transaction from step 3.*

Mesh Server returns: *The transaction identifier after broadcasting.*

Purpose: Broadcast the transaction to the DLT network.

Comparison of Transaction Lifecycles:

Stage Overledger Coinbase Mesh

1. Creation *Prepare (combines intent & formatting) Payloads (creates unsigned tx)* ***2. Verification*** *Implicit in user's signing software Parse (explicit verification step)* ***3. Signing Offline*** *(outside the API flow)* ***Offline*** *(outside the API flow)* ***4. Finalization*** *Handled within Execute Combine (attaches signature)*

5. Broadcast *Execute Submit*

Key Insight: Mesh provides a more granular, explicit, and auditable transaction construction process. This is powerful for wallet and custody providers

who need to guarantee what is being signed. Overledger's process is simpler for application developers who trust the platform to correctly generate

the transaction payload.

5. Interoperability and Security

Interoperability Mechanism

Overledger: *Achieves interoperability at the* ***application layer*** *through a centralized gateway. It's a hub-and-spoke model where Overledger is*

the hub.

Mesh: *Aims for interoperability at the* ***node layer****. It promotes a common interface that any node can adopt, enabling peer-to-peer*

interoperability between any Mesh-compliant applications and nodes.

Security Model

Overledger: *Security is centralized. Users must trust Quant's implementation, infrastructure security, and the correctness of its transaction*

preparation logic. The use of OAuth 2.0 provides a robust, standard authentication framework.

Mesh: *Security is decentralized and the responsibility of the implementer. The user must trust the operator of the specific Mesh*

implementation they are using. The lack of a standard authentication protocol in the spec means security can be inconsistent across different

implementations.

6. Synthesis and Feasibility of a Unified Connector

Architectural Synergy and Divergence

The core synergy is that both platforms use REST APIs and JSON data formats to abstract DLTs. However, their architectures diverge significantly:

one is a product, the other a protocol.

Overledger sells the destination (easy DLT access). Mesh provides a map (a standard way to build the road).

Feasibility of Mapping Overledger to Mesh

Creating a connector where Overledger calls a Mesh-compliant endpoint is **technically feasible but fraught with significant challenges.** This would

> *involve mapping Overledger's API calls to the Mesh API sequence.*
>
> ***Mapping Overledger API -> Mesh API:***
>
> ***Overledger Mesh API Sequence Feasibility & Challenges***
>
> ***Get Balance***

POST /account/balance **High:** Straightforward mapping.

> ***Get*** *POST*
>
> ***Transaction*** */block/transaction*

Medium: Mismatch. Overledger uses transactionId alone; Mesh requires block_identifier and transaction_identifier, making it less efficient and requiring extra lookups.

> ***Low to Medium:*** *The most difficult part. Overledger's high-level intent (type: 'Payment') must be translated into Mesh's low-level operations. For smart contracts, the reference documentation explicitly notes this is "poorly documented" and a "significant implementation hurdle," representing the highest risk.*
>
> *POST*
>
> ***Execute*** */construction/combine* ***Transaction*** *-> POST*
>
> */construction/submit*

High: Feasible. After the client signs the payload from the payloads step, Overledger could use the signature to call combine and then submit.

ISO Standards Context

*Viewing both platforms through the lens of **ISO/TC 307** standards provides a valuable perspective:*

ISO 22739 (Vocabulary): *Both platforms attempt to create a standard vocabulary, but neither explicitly adheres to ISO 22739. Adopting it*

could enhance clarity.

ISO 23257 (Reference Architecture):

Overledger's gateway model fits the concept of an "Application Layer" service that interacts with underlying DLTs.

Mesh's model aligns more closely with standardizing the "DLT Node" interface itself, a fundamental component of the reference

architecture.

Standardization Path: *Mesh's open-source, specification-first approach is more philosophically aligned with the formal, consensus-driven*

process of ISO. Overledger is a proprietary product.

7. Conclusion and Strategic Recommendation

*A connector to allow Overledger to use third-party Mesh implementations is **feasible for basic operations (balance checks, simple transfers) but***

high-risk and complex for advanced functionality like smart contracts. *The ambiguity in the Mesh specification for contract calls and the*

operational overhead of the Mesh model (requiring a hosted node) are significant deterrents.

Recommendation:

*1. **Proceed with Caution:** If a connector is pursued, it should be done in a phased approach. Start with a Proof-of-Concept for simple, well-*

defined functions like token transfers on a single DLT.

*2. **Prioritize Clarification:** Direct engagement with the Coinbase Mesh developer community is essential to resolve the critical ambiguities*

around smart contract payload construction and the conflicting information regarding external node connectivity.

*3. **Evaluate Strategic Alternatives:** Given the friction and immaturity of the Mesh ecosystem, a more viable long-term strategy for Quant may be*

*to **selectively open-source key components of the Overledger platform.** By creating a proprietary but open standard (e.g., an "Overledger*

Connect" SDK), Quant could build a developer ecosystem around its own battle-tested technology, offering a more flexible and robust solution

without the constraints and ambiguities of the Mesh standard.

Comparative Analysis: Quant Overledger vs. Coinbase Mesh

This document provides a detailed technical comparison between the Quant Overledger platform and the Coinbase Mesh specification. The analysis

covers architectural philosophies, data models, API functionality, and transaction workflows, culminating in a blueprint for a potential integration

connector.

Key Insights

***Philosophical Divide:** Overledger is a centralized, high-abstraction **Platform-as-a-Service (PaaS)** that hides blockchain complexity. Mesh is*

*a decentralized, open-source **specification** that standardizes developer interaction with nodes they control.*

***Abstraction vs. Control:** Overledger offers simplicity through abstraction (e.g., tokenName), but less control. Mesh offers granular control via*

its Operations model but demands more technical work from the developer.

***Workflow Alignment:** Both platforms use a Prepare -> Sign -> Execute pattern, making a conceptual mapping feasible. However, the*

data required at each step differs significantly.

***Smart Contract Interaction:** This is the most complex area to map. Overledger uses structured, intent-based requests (e.g.,*

PrepareMint...), while Mesh uses a generic, pass-through /call and /construction/payloads API that requires the developer to

build the raw interaction data.

***Infrastructure:** Overledger is a managed service. A Mesh implementation requires the developer to run and maintain the implementation*

service and a corresponding full blockchain node.

1. Architectural Comparison

> *The fundamental difference between Overledger and Mesh lies in their architectural approach to blockchain interoperability.*

Feature

Model

Quant Overledger

Centralized Gateway (PaaS)

Coinbase Mesh

Decentralized Specification (Middleware)

Analysis

Overledger acts as a single point of entry. Mesh is a standard that developers implement and run themselves.

Feature

Abstraction

Quant Overledger

High-level Abstraction: Hides DLT specifics behind intent-based APIs (e.g., "mint token").

Coinbase Mesh

Standardization Layer: Standardizes the format of requests to a node but requires developers to construct low-level operation details.

Analysis

Overledger prioritizes ease of use; Mesh prioritizes standardization and control.

Managed by Quant: Infrastructure Developers interact

> *endpoint.*
>
> *Overledger has lower operational overhead for the developer. Mesh*

and a full blockchain node, typically gives the developer full ownership in a co-located Docker container. infrastructure.

Provide a single, unified API to access

Primary Goal many blockchains without running any nodes.

Provide a universal specification for any application to interact with any blockchain node in a consistent way.

Overledger sells a service. Mesh provides an open-source standard.

2. Data Model Mapping

This section maps the core data entities between the two platforms. A direct one-to-one mapping is often not possible due to differing abstraction

levels.

Concept

Quant Overledger Coinbase Mesh Representation Representation

Mapping Analysis

Account

A simple string accountId. {"accountId": "0x..."}

An AccountIdentifier object.

{"address": "0x...", "sub_account": }

Mappable. Overledger's accountId maps directly to Mesh's address. Mesh's sub_account (for memos/tags) is an enhancement.

Network

Token

A location object specifying technology and network. {"technology": "Ethereum", "network": "Goerli"}

A first-class, named entity (tokenName) within a request. "tokenName": "MyQRC20Token"

A NetworkIdentifier

object with blockchain **Directly Mappable.** The concepts are identical, only {"blockchain": the field names differ.

"ethereum", "network": "goerli"}

A Currency object within

an Amount. **Complex Mapping.** Overledger's high-level

{"symbol": "USDC", tokenName must be resolved to a specific contract "decimals": 6, address to create a Mesh Currency object. This

{"contractAddress": requires a lookup service. "0x..."}}

> *Returned as part of an API response, with chain-specific details nested in nativeData.*

A standardized Transaction object containing a list of Operation objects.

High Complexity. Overledger's nativeData is a "black box" of chain-specific data. Mesh's Operation model is a structured, standardized representation of transaction effects (e.g., balance changes). Mapping requires parsing nativeData to build a list of Operations.

Smart Contract Call

Write: Part of the Prepare... schema.

Read: Inferred to be similar.

Write: Constructed as a **High Complexity.** The biggest challenge.

list of Operation objects Overledger's structured request (e.g., mint) must be sent to deconstructed into a generic Mesh Operation. /construction/payloads. Conversely, a generic Mesh Operation must be **Read:** A generic request analyzed to infer the intent for a structured

to the /call endpoint. Overledger call.

> *3. API Functionality Comparison*

Functionality Quant Overledger

Coinbase Mesh Endpoint(s)

Comparison & Gaps

Get Account *Not specified, but assumed* POST /account/balance

Mesh's API is explicit and supports specifying currency (token) for the balance check.

Functionality Quant Overledger

Coinbase Mesh Endpoint(s)

Comparison & Gaps

Get Block Data

Not specified, but assumed POST /block

Both platforms support retrieving blocks by hash or height.

Get Transaction

Read Smart Contract

Endpoint likely takes a single transaction ID.

Not specified, but inferred.

POST /block/transaction

POST /call

Gap/Difference: Mesh requires both a block_identifier and transaction_identifier, which is less efficient than a direct lookup by transaction hash.

Gap/Difference: Mesh's /call is generic and poorly documented, making it hard to use without deep chain knowledge. Overledger's approach is likely more structured and user-friendly.

Prepare Transaction

Overledger's endpoint is high-level and POST /v2/preparation/... intent-based. Mesh's endpoint is low-level,

> *requiring a detailed list of operations.*

Submit Transaction

POST POST /v2/execution/transaction/construction/submit

Functionally equivalent; both broadcast a signed transaction.

Key/User POST /authorise/* **Management** endpoints

> ***Unique to Overledger.*** *Mesh is*

None unopinionated about key management and expects the client to handle it entirely.

Compliance Testing

None

mesh-cli check:* commands

Unique to Mesh. The CLI tool provides a standardized way to verify that a Mesh implementation is correct.

NFT Support suppor standard is explicitly *None*

Gap in Mesh. NFTs are not a first-class citizen in the Mesh specification and would require non-standard use of the metadata field.

> *4. Transaction/Execution Workflow Comparison*
>
> *Both platforms follow a similar three-stage security pattern, but the implementation details and data flow are distinct.*

Step Quant Overledger Workflow Coinbase Mesh Workflow

1. Prepare

2. Sign

POST /v2/preparation/transactions/supply POST /construction/payloads

Client sends a high-level, intent-based request Client sends a low-level request containing a list of (e.g., mint 500 tokens to an address). operations (e.g., call contract X, function Y, with Overledger returns a requestId and chain- params Z). Mesh returns an unsigned_transaction specific nativeData to be signed. and a list of signing_payloads.

Client-Side Client-Side

The client uses their key management solution The client uses their own wallet/HSM to sign the (e.g., Authorise API, Overwallet) to sign the signing_payloads received from the Mesh nativeData received from Overledger. implementation.

Combine *(Implicit in Execute step)*

POST /construction/combine

Client sends the unsigned_transaction and the generated signatures to the Mesh implementation. It returns a final, network-ready signed_transaction.

4. Execute

POST /v2/execution/transaction

Client sends the signedTransaction and the original requestId to Overledger, which then broadcasts it to the target DLT.

POST /construction/submit

Client sends the signed_transaction received from the combine step to the Mesh implementation for broadcasting.

> ***Analysis:*** *Mesh's workflow is more verbose with the explicit /construction/combine step, which offers greater transparency and offline*
>
> *capabilities. Overledger's workflow is more streamlined for the developer by abstracting away the low-level construction and combination logic.*
>
> *5. Translation Logic Blueprint*
>
> *This blueprint outlines the rules required to build a software connector that translates between the two APIs.*
>
> ***Scenario 1: Making Overledger Act Like a Mesh Implementation***
>
> *This connector would sit in front of the Quant Overledger API and expose a compliant Coinbase Mesh interface.*
>
> ***Connector Logic:***

1. **On POST /network/list or /network/options:**

> *The connector must query Overledger for its supported DLTs.*
>
> *It will then translate Quant's location format into Mesh's NetworkIdentifier format and return the list.*

2. **On POST /construction/payloads:**

> ***Analyze Operations:*** *The connector must parse the incoming operations array from the Mesh request to determine the user's intent*
>
> *(e.g., token transfer, contract mint, generic call).*
>
> ***Translate to Overledger Prepare:*** *Based on the intent, the connector constructs the appropriate Overledger request body (e.g.,*
>
> *PrepareMintTransactionRequestSchema). This requires a lookup service to map a contract address from a Mesh Currency*
>
> *object to a Quant tokenName.*
>
> ***Call Overledger:*** *The connector calls the relevant Overledger /v2/preparation/* endpoint.*
>
> ***Format Response:*** *The connector takes the nativeData from the Overledger response and formats it into the signing_payloads*
>
> *required by Mesh. It must also store the requestId from Overledger, perhaps embedding it in the metadata field of the Mesh*
>
> *response for later retrieval.*

3. **On POST /construction/combine:**

> *The connector receives the unsigned_transaction (which it likely generated or stored in the previous step) and the signatures*
>
> *from the client.*
>
> *It combines these to produce the final signed transaction string, which it returns in the Mesh signed_transaction format.*

4. **On POST /construction/submit:**

Retrieve requestId: *The connector receives the signed_transaction. It must retrieve the original Overledger requestId that*

was stored or embedded during the payloads step.

Call Overledger Execute: *The connector calls Overledger's /v2/execution/transaction endpoint, providing the*

signedTransaction and the retrieved requestId.

Format Response: *The response from Overledger is then formatted into Mesh's TransactionIdentifierResponse.*

Scenario 2: Making a Mesh Implementation Act Like Overledger

This connector would sit in front of a Coinbase Mesh implementation and expose a Quant Overledger-compatible API.

Connector Logic:

1. **On POST /v2/preparation/transactions/supply:**

Analyze Overledger Request: *The connector parses the high-level intent from the Overledger request (e.g., "mint", tokenName,*

amount).

Translate to Mesh Operations: *The connector must translate this intent into a valid list of operations for the Mesh*

/construction/payloads endpoint. This requires a lookup service to convert the tokenName into a contract address, function

signature, and parameters.

Call Mesh Payloads: *The connector calls the /construction/payloads endpoint of the target Mesh implementation.*

Format Response: *The connector takes the signing_payloads from the Mesh response and formats it into the nativeData*

structure expected by the Overledger client. It must also generate its own requestId and store a mapping between this ID and the

unsigned_transaction returned by Mesh for the next step.

2. **On POST /v2/execution/transaction:**

Retrieve Unsigned Data: *The connector receives the signedTransaction and requestId from the client. It uses the requestId*

to look up the unsigned_transaction that was stored from the previous step.

Call Mesh Combine & Submit:

1. The connector calls the Mesh /construction/combine endpoint, providing the stored unsigned_transaction and the

client's signedTransaction (as the signature).

2. It takes the resulting signed_transaction from the combine step and immediately calls the Mesh

/construction/submit endpoint with it.

Format Response: *The transaction_identifier from the Mesh submit response is formatted and returned to the Overledger*

client.

Project Synthesis Report: DLT Interoperability Analysis and Connector Development

1. Project Goal

The primary objective of this project was to conduct a comprehensive evaluation of the Quant Overledger and Coinbase Mesh data models and API

specifications. Based on this analysis, the goal was to design and implement a functional proof-of-concept connector to translate between the two

platforms, demonstrating a practical path toward greater blockchain interoperability.

2. Methodology

The project was executed through a systematic, multi-phase research and development process:

1. ***Documentation Analysis:*** *A thorough review was conducted on all provided documentation for Quant Overledger and Coinbase Mesh. This*

included API references, technical architecture documents, and investigative reports.

2. ***Independent Research:*** *The initial analysis was augmented by independent research into the ISO/TC 307 standards for blockchain and DLTs,*

providing context for creating a formal, standards-compliant data model.

*3. **Comparative Synthesis:** The findings were synthesized into a detailed comparative analysis that identified the core philosophies,*

architectural differences, feature gaps, and potential synergies between the two platforms.

*4. **Model & Architecture Design:** Leveraging the insights from the analysis, a new unified data model was proposed, and a technical*

architecture for a middleware connector was designed.

*5. **Proof-of-Concept Implementation:** The designs were realized in a functional proof-of-concept, culminating in a comprehensive project*

showcase website.

3. Key Findings

The comparative analysis revealed that while both platforms share a common goal of simplifying DLT interaction, their approaches are fundamentally

different.

***Philosophical Divide:** Quant Overledger operates as a **managed API gateway**, offering interoperability-as-a-service and abstracting all*

*underlying node infrastructure. Coinbase Mesh is an **open-source specification**, standardizing the direct interface to self-hosted nodes.*

***Core Synergy:** The most significant area of alignment is the conceptual Prepare -> Sign -> Execute transaction lifecycle. This shared*

pattern provides a robust foundation for mapping write operations between the systems.

***Key Differences:** The platforms diverge significantly in abstraction, features, and operational models.*

Aspect

Abstraction Level

Quant Overledger

High: Uses intent-based, abstract APIs (e.g., "Payment").

Coinbase Mesh

Low: Uses granular, operational APIs (e.g., a "TRANSFER" operation).

Aspect Quant Overledger Coinbase Mesh

Infrastructure Managed: No nodes required for

Self-Hosted: Requires the user to run a full DLT node per implementation.

Features

Smart Contracts

Includes value-added services like built-in webhooks.

Simplified, purpose-built endpoints for interaction.

Specification is stateless and lacks features like push notifications.

Generic /call and /payloads endpoints that are powerful but complex and poorly documented.

4. Proposed Unified Data Model

*To harmonize the two systems, a **Unified Data Model for Blockchain and DLT Interaction** was proposed, designed for potential submission as an*

ISO standard. This model leverages the strengths of both platforms:

***Foundation:** It adopts the granular, explicit Operation-based transaction structure from Coinbase Mesh as its core. This ensures that all*

state changes are clear and auditable.

***Extensibility:** It integrates the high-level contextual data from platforms like Overledger through a formally defined, namespaced metadata*

schema. This allows platform-specific data (e.g., requestId, urgency) to be carried through the system in a structured way without

compromising the core standard.

5. Connector Implementation

*A functional **Quant-to-Coinbase Mesh Connector** was developed to validate the proposed model.*

***Technical Architecture:** The connector is designed as a stateless middleware service. It intercepts a high-level Quant Overledger API request*

and translates it into the corresponding low-level Coinbase Mesh request.

***Core Functionality:** The translation logic deconstructs Overledger's intents into Mesh's atomic operations. For example:*

A single Quant Payment request is decomposed into two Mesh TRANSFER operations: a debit from the origin account and a credit to

the destination account.

The related_operations field in Mesh is used to link these two operations, preserving the atomicity of the original intent.

The connector respects the client-side security model by handling the transaction "preparation" step and returning an unsigned payload

to the client for signing.

Example Translation Logic (translator.js):

// Deconstruct a single Quant 'Payment' into two atomic Mesh 'TRANSFER' operations

```
operations.push({

operation_identifier: { index: 0 },

type: 'TRANSFER',

account: { address: origin[0].addressId },

amount: {

value: `-${totalAmount.toString()}`, // Debit from origin

currency: currency,

},

});

operations.push({

operation_identifier: { index: 1 },

related_operations: [{ index: 0 }], // Link to the debit operation

type: 'TRANSFER',

account: { address: destination[0].destinationId },

amount: {

value: totalAmount.toString(), // Credit to destination

currency: currency,

},

});
```

6. Final Deliverable: Project Showcase Website

The final project output is a comprehensive, interactive ***project showcase website****. This website serves as a single, integrated deliverable that*

consolidates all project phases and artifacts:

Analysis & Documentation: *It hosts the detailed documentation from the project, including the Comparative Analysis, the Unified Data Model*

proposal, and the Connector's Technical Architecture.

> ***Functional Proof-of-Concept:*** *It features a live, interactive demo of the Quant-to-Mesh connector. Users can input valid Quant Overledger*
>
> *JSON requests and see the real-time translation into the Coinbase Mesh format, validating the core logic of the implementation.*
>
> *This integrated showcase effectively demonstrates the project's journey from theoretical analysis to practical application.*
>
> ***Conclusion***
>
> *The project successfully achieved all stated objectives. The detailed analysis of Quant Overledger and Coinbase Mesh provided critical insights into*
>
> *the challenges and opportunities in DLT interoperability. These insights led to the design of a robust Unified Data Model and the implementation of a*

functional connector, proving the technical feasibility of harmonizing high-level, managed DLT gateways with low-level, open-standard interfaces. The

final showcase website delivers a complete summary of the project's findings and a tangible demonstration of its success.

Overledger Technical Overview & API Reference

This document provides a comprehensive technical overview of the Quant Overledger platform. It details the core architecture, authentication

mechanisms, data models, and a complete API reference for interacting with distributed ledger technologies (DLTs) through a single, standardized

interface.

1. Core Concepts and Architecture

Overledger is an operating system for blockchains that enables interoperability between different DLTs and existing networks. It achieves this without

adding another consensus layer, providing a simple yet powerful API gateway to interact with a wide array of supported blockchains.

Key Architectural Principles:

Abstraction Layer: *Overledger abstracts the complexities of individual DLTs. Developers interact with a single, unified API, and Overledger*

handles the protocol-specific communication with the target blockchain (e.g., Ethereum, Bitcoin, XRP Ledger).

Two-Step Transaction Model: *To ensure security and non-custodial control, Overledger employs a two-step process for submitting*

transactions.

1. ***Preparation:*** *The user submits transaction details to a /preparation endpoint. Overledger validates the request and returns a DLT-*

specific, unsigned transaction payload.

2. ***Execution:*** *The user signs this payload offline using their private keys. The signed transaction is then submitted to an /execution*

endpoint, which broadcasts it to the target DLT.

Standardized Data Models: *All data, whether for transactions, accounts, or blocks, is presented in a standardized JSON format, regardless of*

the underlying DLT.

2. Authentication and Authorization

*Access to the Overledger API is secured using **OAuth 2.0**. All requests must include a valid Bearer token in the Authorization header.*

2.1. Obtaining API Credentials

*1. **Sign Up:** Register for an account on the Quant Developer Portal.*

*2. **Create an Application:** Within your developer dashboard, create a new application (mApp).*

*3. **Generate Keys:** Upon application creation, you will be issued a clientId and a clientSecret. Store the clientSecret securely as it*

cannot be retrieved again.

2.2. Generating a JWT Bearer Token

A JWT is generated by making a POST request to the OAuth2 token endpoint using your clientId and clientSecret.

***Endpoint:** POST /oauth2/token*

***Authentication:** Basic Auth (clientId as username, clientSecret as password).*

***Request Body:** The request must be of type application/x-www-form-urlencoded.*

Parameter Type Required Description

grant_type

client_id

string Yes

string Yes

The grant type. Must be set to client_credentials.

Your application's unique Client ID.

client_secretstring Yes Your application's secret.

Example cURL Request:

*curl -X POST https://api.overledger.io/oauth2/token (https://api.overledger.io/oauth2/token) *

*-H "Content-Type: application/x-www-form-urlencoded" *

-d "grant_type=client_credentials&client_id=YOUR_CLIENT_ID&client_secret=YOUR_CLIENT_SECRET"

Example Success Response:

{

"access_token": "eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2IkIiA6ICJQS1d...",

"expires_in": 300,

"refresh_expires_in": 0,

"token_type": "Bearer",

"not-before-policy": 0,

"scope": "email profile"

}

Important: *The access_token returned from this call must be included in the Authorization header for all subsequent API requests as*

Bearer <access_token>.

3. Core Data Models

Overledger uses standardized data models to represent DLT concepts.

3.1. Location Object

The location object is a fundamental structure used in virtually all API calls to specify the target DLT for a given operation.

Field Type Description

technologystring The name of the distributed ledger technology (e.g., Ethereum, Bitcoin, XRP Ledger).

network

The specific network of the DLT (e.g., Ethereum Goerli Testnet, Bitcoin Mainnet, XRPL Testnet).

Example location Object:

{

"technology": "Ethereum",

"network": "Ethereum Goerli Testnet"

}

3.2. Transaction Models

Transaction Request (Simplified)

Used for preparing transactions.

Field type

urgency

location

origin

Type Description

string The type of transaction (e.g., Payment, Contract Invoke).

string The desired transaction speed (e.g., Normal). Mapped to DLT-specific fee/gas settings. object The target DLT network. See Location Object.

array An array of sender account objects, each containing an accountId.

destinationarray An array of recipient objects, each containing an accountId and a payment object with amount

Transaction Response

The detailed view of a transaction as returned by search endpoints.

Field Type Description

Field Type Description

transactionIdstring The unique hash/identifier of the transaction on the DLT.

status

fee

timestamp

...

object An object containing the transaction value (e.g., CONFIRMED). object An object containing the amount and unit of the fee paid. string The ISO 8601 timestamp of the transaction.

Other DLT-specific fields.

4. Transaction and Smart Contract Flows

4.1. Standard DLT Payment Flow

This two-step flow ensures that private keys never leave the user's control.

Step 1: Prepare the Transaction *Submit the transaction details to Overledger to receive a DLT-native, unsigned transaction payload.*

Endpoint: *POST /v2/preparation/transaction*

Action: *Provide location, origin, destination, and other payment details.*

Result: *Overledger returns a requestId and dltData, which contains the unsigned transaction payload.*

```
// Example Response from /v2/preparation/transaction

{

"requestId": "a1b2c3d4-e5f6-7890-g1h2-i3j4k5l6m7n8",

"dltData": [

{

"transaction": "0xf86c098504a817c80082520894c52f7904..."

}

],

"nativeData":

}
```

Step 2: Sign and Execute the Transaction *Sign the dltData.transaction value offline with the appropriate private key.*

Endpoint: *POST /v2/execution/transaction*

Action: *Submit the requestId from the preparation step and the signed transaction payload.*

Result: *Overledger validates the signature and broadcasts the transaction to the specified DLT network. It returns an*

overledgerTransactionId for tracking.

4.2. Smart Contract Interaction Flow

Interacting with smart contracts follows a similar prepare-and-execute pattern.

*1. **Prepare Smart Contract Call:** Use the POST /v2/preparation/smartcontract endpoint. Specify the contract's address or name, the*

functionName, and any inputValues. Overledger returns an unsigned payload.

*2. **Sign and Execute:** Sign the payload and submit it to the POST /v2/execution/smartcontract endpoint.*

*3. **Query Smart Contract State:** Use the POST /v2/search/smartcontract/ endpoint to read data from smart contracts without requiring a*

transaction. Provide the contract details and the function you wish to query.

5. Complete API Reference

All requests require an Authorization: Bearer <JWT> header.

5.1. TRANSACT API - Making Payments

Prepare a DLT Transaction

POST /v2/preparation/transaction

Prepares a transaction for signing by validating the request and returning a DLT-specific payload.

Request Body:

{

"location": {

"technology": "Ethereum",

"network": "Ethereum Goerli Testnet"

},

"type": "Payment",

"urgency": "Normal",

"requestDetails": {

"origin": [

{

"accountId": "0x...YourSenderAddress"

}

],

"destination": [

{

"accountId": "0x...RecipientAddress",

"payment": {

"amount": "10000000000000000",

"unit": "wei"

}

}

]

}

}

Execute a Signed DLT Transaction

POST /v2/execution/transaction

Executes a previously prepared and signed transaction.

Request Body:


```
{
"requestId": "a1b2c3d4-e5f6-7890-g1h2-i3j4k5l6m7n8",
"signed": "0xf86e098504a817c80082520894c52f7904..."
}
```


Success Response:


```
{
"overledgerTransactionId": "a4d33b5c-d788-4613-b6d3-96b5c005f3d3",
"location": {
"technology": "Ethereum",
"network": "Ethereum Goerli Testnet"
},
"status": "PENDING"
}
```

5.2. SEARCH API - Reading DLT Data

Search for a Transaction

POST /v2/search/transaction

Retrieves details for a specific transaction using its hash.

Request Body:

```
{
"transactionId": "0x123...abc",
"location": {
"technology": "Ethereum",
"network": "Ethereum Goerli Testnet"
}
}
```

Search for an Address Balance

POST /v2/search/address/balance/

Retrieves the balance of a specific address. The addressId is specified in the URL path.

Request Body (Optional):

```
{
```

"location": {

"technology": "Ethereum",

"network": "Ethereum Goerli Testnet"

}

}

Success Response:

{

"location": {

"technology": "Ethereum",

"network": "Ethereum Goerli Testnet"

},

"balances": [

{

"unit": "ETH",

"amount": "5000000000000000000"

}

],

"accountId": "0x...YourAddress"

}

Search for a Block

POST /v2/search/block/

Retrieves details for a specific block by its number or hash (blockId).

Request Body:

{

"location": {

"technology": "Ethereum",

"network": "Ethereum Goerli Testnet"

}

}

Search a Smart Contract

POST /v2/search/smartcontract

Queries a read-only function of a smart contract.

Request Body:

{

```
"location": {
"technology": "Ethereum",
"network": "Ethereum Goerli Testnet"
},
"requestDetails": {
"destination": [{
"smartContract": {
"address": "0x...",
"function": { "name": "balanceOf", "inputs": [{"type": "address", "value": "0x..."}] }
}
}]
}
```

}

5.3. WEBHOOKS API - Real-time Updates

Create Account Webhook

POST /v2/webhook/subscription

Subscribes to notifications for transactions affecting a specific account.

Request Body:

```
{
"type": "Account",
"location": {
"technology": "Ethereum",
"network": "Ethereum Goerli Testnet"
},
"callbackUrl": "https://yourapi.com/webhook-receiver",
"accountId": "0x...YourAddressToMonitor"
}
```

Success Response:

```
{
"webhookId": "f9a4b2d1-8e7f-4c6a-9b3d-0f5e8a7b6c5d",
"subscriptionId": "e1a2b3c4-d5e6-f7g8-h9i0-j1k2l3m4n5o6"
}
```

Retrieve a List of Webhooks

GET /v2/webhook/subscriptions

Returns a list of all active webhook subscriptions for your application.

Retrieve Information about a Webhook

GET /v2/webhook/subscription/

Fetches the details of a specific webhook subscription using its webhookId.

Technical Architecture: Quant-to-Coinbase Mesh Connector

1. Introduction

This document outlines the technical architecture and design for a connector service that bridges the Quant Overledger API with Coinbase Mesh

implementations. The primary goal is to enable applications built on the Overledger API standard to interact with distributed ledger technologies (DLTs)

that are only accessible via a Mesh-compliant interface.

The architecture is designed to translate Overledger's managed, high-level API calls into the corresponding low-level, self-hosted operations defined

by the Coinbase Mesh specification. It addresses key feature gaps, including authentication and webhooks, and proposes a phased implementation

plan to mitigate risks identified in the initial analysis.

2. System Architecture Overview

The connector operates as a middleware component, intercepting API calls intended for Overledger and translating them for a target Mesh

implementation. This architecture assumes that for each target DLT, a corresponding Coinbase Mesh service and a full blockchain node are deployed

and managed alongside the connector.

2.1. Component Diagram

The following diagram illustrates the two distinct interaction models. The connector's role is to bridge the "Gateway Model" (which clients are built for)

with the "Standardization Model" (which the target DLT exposes).

2.2. Component Responsibilities

Component

Client Application

Responsibility

The end-user system built against the Quant Overledger API specification.

Technical Implementation Notes

Unaware of the underlying Mesh translation. It sends standard Overledger API requests.

Quant-Mesh Connector

The core translation engine. A

stateless service for transaction Deployed as a containerized microservice (e.g., Node.js, processing and a stateful service Python, Go).

for webhook polling.

> *A reverse proxy placed in front of NGINX, AWS API Gateway, or similar. It validates the*
>
> *the connector to manage Overledger-style JWT and can inject API keys if required by security. the Mesh endpoint.*

Endpoint Configuration

A service or configuration file for mapping Overledger location objects to Mesh URLs.

Can be a simple YAML/JSON file or a dynamic service discovery mechanism (e.g., Consul).

> *A stateful background worker responsible for simulating webhook functionality.*
>
> *The target DLT-specific service implementing the Coinbase Mesh OpenAPI specification.*

A separate process or thread that queries Mesh for transaction statuses and POSTs updates to client-registered URLs. Requires a small database (e.g., Redis, PostgreSQL) to store subscriptions and job states.

Hosted in a container, co-located with its full node as recommended by the Mesh standard.

3. API Interaction and Data Transformation

The core function of the connector is the bi-directional translation of API requests and responses. The Prepare -> Sign -> Execute flow is the

foundation for this mapping.

3.1. Transaction Preparation Flow (/v2/preparation/transaction)

1. ***Client -> Connector:*** *The client sends a POST /v2/preparation/transaction request formatted for the Overledger API.*

2. ***Connector:***

Receives the Overledger request.

Uses the location object to look up the target Mesh implementation URL from the ***Endpoint Configuration****.*

Transforms the request body into a POST /construction/payloads request for Mesh.

3. ***Connector -> Mesh:*** *Forwards the translated request to the target Mesh endpoint.*

4. ***Mesh -> Connector:*** *Mesh returns a ConstructionPayloadsResponse containing unsigned transaction data.*

5. ***Connector:***

Translates the Mesh response back into an Overledger PrepareTransactionResponse.

The raw unsignedTransaction data from Mesh is embedded directly.

6. ***Connector -> Client:*** *Sends the final Overledger-formatted response to the client for signing.*

3.2. Data Mapping: Overledger Preparation to Mesh Payloads

Overledger (/preparation/transaction)

location

Mesh (/construction/payloads)

network_identifier

Transformation Logic

The connector uses the location object to select the correct Mesh endpoint and construct the network_identifier.

Overledger (/preparation/transaction)

type (e.g., "Payment")

origin[].address

Mesh (/construction/payloads)

operations (array of Operation)

operations[0].account

Transformation Logic

An Overledger "Payment" maps to a single Mesh Operation of type: "TRANSFER".

Mapped to the account.address field in the Mesh Operation.

The amount is mapped directly. The destination[].payment.amountoperations[0].amount.value currency object is constructed based on

> *the DLT specified in location.*

destination[].payment.unit

fee.price / fee.limit

operations[0].amount.currency.symbolMapped to the currency object.

Overledger's explicit fee parameters may not map directly. Mesh often calculates this. If provided, they can be passed in metadata as a suggestion for the Mesh implementation.

Example Transformation (Request):

Overledger Request Body Snippet:


```
{
"location": {
"technology": "Ethereum",
"network": "Goerli Testnet"
},
"type": "Payment",
"destination": [{
"payment": { "amount": "10000000000000000", "unit": "wei" }
}]
}
```


Translated Mesh Request Body:


```
{
"network_identifier": {
"blockchain": "ethereum",
"network": "goerli"
},
"operations": [{
"operation_identifier": { "index": 0 },
"type": "TRANSFER",
"account": { "address": "0x...OriginAddress..." },
"amount": {
```

"value": "-10000000000000000",

"currency": { "symbol": "ETH", "decimals": 18 }

}

}, {

"operation_identifier": { "index": 1 },

"related_operations": [{ "index": 0 }],

"type": "TRANSFER",

"account": { "address": "0x...DestinationAddress..." },

"amount": {

"value": "10000000000000000",

"currency": { "symbol": "ETH", "decimals": 18 }

}

}],

"public_keys": [{

"hex_bytes": "...",

"curve_type": "secp256k1"

}]

}

3.3. Transaction Execution Flow (/v2/execution/transaction)

This flow is simpler as it primarily involves passing signed data.

1. **Client -> Connector:** Client sends a POST /v2/execution/transaction request containing the signed transaction data from the

previous step.

2. **Connector:**

> *Looks up the target Mesh endpoint via the location.*
>
> *Constructs a POST /construction/submit request. The signed_transaction field is populated with the data provided by the*
>
> *client.*

3. **Connector -> Mesh:** Forwards the request to the Mesh implementation.

4. **Mesh -> Connector:** Mesh returns a TransactionIdentifierResponse.

5. **Connector:** Translates the response into an Overledger ExecuteTransactionResponse, mapping transaction_identifier.hash to

> *transactionId.*

6. **Connector -> Client:** Sends the final response.

4. Strategy for Feature Gaps and Infrastructure

4.1. Authentication

Problem: *Overledger uses OAuth2 JWTs. Mesh has no defined authentication standard.*

Solution: *Implement an* ***Authentication Proxy*** *using a tool like NGINX or AWS API Gateway.*

1. The proxy is the public-facing endpoint for the connector.

2. It intercepts all incoming requests and validates the Authorization: Bearer <JWT> header against Quant's public keys.

3. If valid, it forwards the request to the connector service.

4. If the target Mesh implementation requires an API key, the proxy can be configured to inject it into the outgoing request.

Benefit: *This decouples security from the core translation logic, allowing the connector to remain focused on its primary task.*

4.2. Webhooks

Problem: *Overledger provides push-based event notifications via webhooks. Mesh has no such feature and requires polling.*

Solution: *Implement a* ***Webhook Polling Service*** *as a stateful component of the connector solution.*

1. ***Subscription:*** *When a client calls the /v2/webhook/subscription endpoint on the connector, the details (e.g., callbackUrl,*

transactionId) are stored in a persistent database (e.g., Redis).

2. ***Polling:*** *A background worker process periodically queries the database for active subscriptions.*

3. ***Status Check:*** *For each active subscription, the poller calls the relevant Mesh endpoint (e.g., /block/transaction or*

/search/transactions) to get the latest status of the transaction.

4. ***Notification:*** *If a status change is detected, the service formats a standard Overledger webhook payload and sends a POST request to*

the client's callbackUrl.

5. ***De-activation:*** *Once a transaction reaches a final state (confirmed or failed), the subscription is marked as inactive to prevent further*

polling.

4.3. Infrastructure and Deployment

Requirement: *Mesh implementations must be run with co-located DLT full nodes.*

Strategy: *The entire solution should be containerized using Docker and orchestrated with Kubernetes or Docker Compose. A typical*

deployment for one DLT would consist of:

A pod/container for the ***Quant-Mesh Connector*** *service.*

A pod/container for the ***Webhook Polling Service*** *and its database.*

A pod/container running the ***Coinbase Mesh Implementation*** *for the target DLT.*

A pod/container running the ***DLT Full Node****.*

An Ingress/LoadBalancer configured as the ***Authentication Proxy****.*

Error Scenario

Invalid Overledger Request

Detection

Schema validation fails in the connector.

5. Error Handling

Connector Action

Reject the request immediately.

Client Response

400 Bad Request with a descriptive error message.

The location object **Unsupported** does not map to a **location** configured Mesh

endpoint.

Check against the Endpoint Configuration.

422 Unprocessable Entity with message: "The specified DLT/network is not supported."

Mesh API Error

Mesh endpoint returns a Log the full Mesh error. Map the non-2xx status with a Mesh error code and message to Mesh Error object. the Overledger error format.

A corresponding 4xx or 5xx error with a translated message.

TCP/IP or DNS errors **Unavailability** when calling the Mesh

Implement a retry mechanism with exponential backoff for a limited number of attempts.

503 Service Unavailable if retries fail.

6. Phased Implementation Plan

Phase 1: Proof of Concept (PoC) - Core Value Transfer

Objective: *Validate the fundamental translation logic for a single DLT (e.g., Ethereum).*

Scope:

Implement translation for preparation/transaction and execution/transaction for simple value transfers.

Hardcode the Mesh endpoint URL and any required API keys.

No Authentication Proxy or Webhook Polling Service.

Basic error handling and logging.

Success Criteria: *An end-to-end test successfully prepares, signs (offline), and executes a value transfer on a testnet, with the client only*

interacting via Overledger API calls.

Phase 2: Minimum Viable Product (MVP) - Production Readiness

Objective: *Develop a secure, configurable, and deployable connector.*

Scope:

Implement the Authentication Proxy for JWT validation.

Build the dynamic Endpoint Configuration service/loader.

Build and deploy the Webhook Polling Service with a persistent store.

Implement comprehensive, structured logging and robust error mapping.

Containerize the entire application stack for orchestrated deployment.

Success Criteria: *The connector can handle multiple DLTs, is secure, and provides feature parity with Overledger's core transaction and*

notification capabilities.

Phase 3: Extended Functionality - Smart Contracts & NFTs

Objective: *Support complex interactions beyond simple value transfers.*

Prerequisite: This phase is contingent on resolving the ambiguity around Mesh's /construction/payloads endpoint for smart

contracts. *Requires R&D and engagement with the Mesh community.*

Scope:

Develop a generic-to-specific translation module that can build chain-specific smart contract payloads for Mesh from Overledger's

abstract function call definitions.

Investigate and implement a non-standardized mapping for NFT operations, likely using the metadata field.

Success Criteria: *The connector can successfully execute a common smart contract function (e.g., an ERC20 transfer) via the Overledger*

API.

Quant Network Overledger V3: Technical Overview

The Quant Network Overledger platform is engineered as a universal interoperability solution, functioning as a blockchain-agnostic API gateway. It

aims to abstract the complexity of diverse Distributed Ledger Technologies (DLTs), enabling developers to build and deploy multi-DLT applications

(mDApps) through a single, standardized interface.

1. Core Architecture and Interoperability Model

Overledger is architected as an orchestration layer that operates on top of existing blockchains without requiring any modifications to the underlying

protocols. This "operating system" approach allows it to connect and manage workflows across multiple, disparate ledgers.

Architectural Layers *The foundational design of Overledger is based on a multi-layered architecture that separates concerns, from raw transaction*

data to application-level logic.

Layer

Transaction Layer

Messaging Layer

Filtering & Ordering Layer

Application Layer

Function

Interfaces directly with various DLTs, storing transactions that are appended to the native ledger technologies.

Abstracts and retrieves relevant information (transaction data, smart contract state, metadata) from all connected ledgers into a standardized format.

Validates, filters, and orders messages from the Messaging Layer. It establishes connections between transactions across different ledgers to enable cross-chain logic.

Manages application-specific messages that have been validated and ordered, ensuring they conform to the required format and signature requirements of the mDApp.

Modern Architectural Components *The V3 implementation builds upon this foundation with modern, industry-standard components:*

REST API Gateway*: The primary developer interface, built on OpenAPI 3.0 standards. It has shifted functionality from older SDKs to a more*

flexible and universally accessible API-first model.

Remote Connector Gateways (RCGs)*: These gateways connect directly to DLT nodes (e.g., Ethereum, Hyperledger Fabric), handling the*

protocol-specific communication required for transaction processing.

Open Digital Asset Protocol (ODAP)*: Quant is a key contributor to this IETF standard, which is designed to facilitate secure and*

standardized asset transfers between different DLT networks, forming the basis for true cross-chain value exchange.

Conceptual Diagram: Overledger as a Unified Interoperability Layer

2. API Command Structure and Interaction Flow

Interaction with the Overledger V3 API is designed to be RESTful and intuitive. The command structure focuses on a two-part flow: **preparation** of an

*action and **monitoring** for its outcome.*

***Authentication** Access to the API is managed through the **Quant Connect** developer portal. Developers must register an application to obtain a*

clientId and clientSecret, which are used for OAuth 2.0-based authentication with the Overledger platform.

General Workflow

*1. **Obtain Credentials**: Generate clientId and clientSecret from Quant Connect.*

*2. **Authenticate**: Establish an authenticated session with the Overledger API.*

*3. **Prepare a Transaction**: A client application sends a request to a /preparations endpoint. This request contains all necessary details for*

the on-chain action (e.g., the smart contract function to call and its parameters). Overledger validates the request and prepares a protocol-

specific transaction payload.

*4. **Sign and Execute**: The prepared transaction is returned to the client for signing. Overledger provides services for automated key*

management and transaction signing to streamline this step.

*5. **Monitor via Webhooks**: Instead of polling for results, developers configure webhooks. Overledger pushes real-time notifications about*

account activity or smart contract events to a pre-defined callback URL.

3. Key V3 API Endpoints

As of November 2023, all Overledger V3 API endpoints require a /api prefix in their URL path. This update enhances security and maintainability

with full backward compatibility during a transition period. As of January 2024, Webhook V3 endpoints have fully superseded the deprecated V2

subscription and monitoring APIs.

Method Endpoint Path Description

Prepares a transaction to execute a write function on a smart POST contracts/write contract. Supports networks like Ethereum and Hyperledger

POST /api/webhooks/smart-contract-events

POST /api/webhooks/accounts

GET /api/webhooks

GET /api/webhooks/

PATCH /api/webhooks/

DELETE /api/webhooks/

Creates a webhook to monitor for specific events emitted by a smart contract.

Creates a webhook to monitor for incoming or outgoing transactions for a specific blockchain account/address.

Lists all existing webhooks for the application. Retrieves the details of a specific webhook. Updates the configuration of an existing webhook.

Deletes a specified webhook.

4. Data Models and Schemas

The Overledger API uses structured data models to ensure consistency across different DLTs. The most detailed examples are derived from its smart

contract and webhook functionalities.

Smart Contract Interaction Model When reading from or writing to smart contracts on Ethereum-based networks, the API requires parameters to be

defined according to their Solidity types.

Supported Types*:*

Basic*: uint, int (all sizes), string, address, bool, bytes (all sizes)*

Arrays*: Dynamic (uint[]) and Fixed (uint[4]) arrays are supported for all basic types.*

Request Structure: A request to call a smart contract function requires specifying the function name and an array of input parameters, each

with a defined type and value.

Response Structure: The response contains an array of output objects, each specifying the type and the returned value.

Webhook Callback Data Model *Webhook payloads provide structured JSON data detailing the on-chain event.*

Smart Contract Event Webhook Payload:


```
{
"type": "smartContractEvent",
"webhookId": "a1b2c3d4-...",
"location": {
"technology": "Ethereum",
"network": "Ethereum Goerli Testnet"
},
"smartContractEventUpdateDetails": {
"smartContractId": "0x...",
"nativeData": {
"transactionHash": "0x...",
"blockHash": "0x...",
"blockNumber": 1234567,
"address": "0x...",
"data": "0x...",
"topics": ["0x..."]
}
}
}
```


Account Event Webhook Payload:

```
{
"type": "account",
"webhookId": "e5f6g7h8-...",
"accountId": "0x...",
"location": {
"technology": "Ethereum",
"network": "Ethereum Goerli Testnet"
},
"transactionId": "0x..."
}
```

5. Key Insights and Strategic Considerations

Abstraction is the Core Value: Overledger's primary strength is abstracting away protocol-specific complexities. Developers write to one

standardized API, and Overledger handles the translation and execution across supported DLTs.

Event-Driven Architecture: The emphasis on Webhooks (V3) over polling (V2) promotes a modern, efficient, and scalable event-driven

architecture for mDApps.

Documentation Gap: While the platform's capabilities are well-documented conceptually, **detailed technical documentation, including**

comprehensive endpoint specifications, raw API request/response examples (e.g., cURL), and complete data schemas, is not

publicly available. Access to the official **Quant Developer Hub** is required for deep integration work.

Low-Code Integration: The existence of official (though archived) Make and Zapier integrations demonstrates a strategic focus on enabling

low-code and no-code automation, broadening the potential user base beyond expert blockchain developers.

> ***Focus on Enterprise and RWA****: The support for permissioned ledgers like Hyperledger Fabric and the overall architectural design indicate a*
>
> *strong focus on enterprise use cases, such as the tokenization and cross-chain transfer of Real-World Assets (RWAs).*
>
> ***Quant Overledger: Technical Overview***

This document provides a comprehensive technical overview of the Quant Overledger platform, synthesized from the official documentation. It covers

core concepts, API references, data models, and interaction flows necessary for development.

1. Core Concepts

Overledger is a blockchain-agnostic API gateway that enables interoperability between different Distributed Ledger Technologies (DLTs). It provides a

single, standardized interface to interact with multiple blockchains without requiring developers to run their own nodes or write chain-specific code.

Single API for All DLTs: *Overledger abstracts the complexities of individual blockchains (e.g., Bitcoin, Ethereum, XRP Ledger) into a unified*

REST API.

Transaction Interoperability: *It facilitates the creation, signing, and execution of transactions across supported networks.*

Data Querying: *Provides standardized endpoints to search for transactions, addresses, blocks, and smart contract states across different*

ledgers.

Security Model: *The platform is designed so that private keys never leave the client's environment. Transactions are prepared by Overledger,*

signed by the client, and then submitted for execution.

A critical component in every API call is the location object, which specifies the target DLT for the operation.

```
{
"technology": "Ethereum",
"network": "Goerli Testnet"
}
```

2. Authentication and Security

Access to the Overledger API is secured using a two-step OAuth2 Client Credentials flow.

1. ***Obtain API Keys:*** *First, you must generate a clientId and clientSecret from the Quant Developer Portal. These are your long-term*

credentials.

2. **Generate a JWT:** Use your clientId and clientSecret with Basic Authentication to request a short-lived JSON Web Token (JWT) from

the Overledger OAuth2 endpoint.

3. **Authorize API Calls:** Include this JWT as a Bearer token in the Authorization header for all subsequent API requests.

API Endpoint: Retrieve OAuth2 Token

This endpoint exchanges your credentials for a bearer token.

Request

Method: POST

Endpoint: /oauth2/token

Headers:

Content-Type: application/x-www-form-urlencoded

Authorization: Basic <base64_encoded_credentials> (where credentials are clientId:clientSecret)

Form Data:

grant_type: client_credentials

Example Request (curl)

```
curl --location --request POST 'https://api.overledger.io/oauth2/token' \

--header 'Content-Type: application/x-www-form-urlencoded' \

--header 'Authorization: Basic YOUR_BASE64_ENCODED_CREDENTIALS' \

--data-urlencode 'grant_type=client_credentials'
```

Example Success Response


```
{

"access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",

"token_type": "Bearer",

"expires_in": 300,

"scope": "read write",

"clientId": "your-client-id"
```

}

3. Transaction and Smart Contract Interaction Flows

Overledger uses a "prepare-and-execute" model to ensure client-side key security.

A. DLT Value Transfer Flow

1. ***Prepare Transaction:*** *Call the POST /v2/preparation/transaction endpoint with transaction details (origin, destination, amount).*

Overledger returns a requestId and a chain-specific dltData payload.

2. ***Sign Transaction:*** *The client uses their private key to sign the dltData payload offline. This is a critical security step.*

3. ***Execute Transaction:*** *Call the POST /v2/execution/transaction endpoint, providing the requestId and the signed transaction*

payload. Overledger broadcasts the transaction to the specified DLT network.

B. Smart Contract Interaction Flow

Interacting with smart contracts follows a similar prepare-and-execute pattern, but the preparation step includes the function name and parameters to

be invoked.

4. API Reference

All requests require the Authorization: Bearer <JWT> header. The base URL for the API is https://api.overledger.io (https://api.overledger.io).

4.1 TRANSACT API

Prepare a DLT Transaction for Signing

Endpoint: *POST /v2/preparation/transaction*

Description: *Prepares a transaction for a value transfer on a specific DLT.*

Request Body

{

```
"location": {
"technology": "Ethereum",
"network": "Goerli Testnet"
},
"type": "VALUE_TRANSFER",
"urgency": "NORMAL",
"requestDetails": {
"origin": [
{
"originId": "0x...YourSourceAddress"
}
],
"destination": [
{
"destinationId": "0x...RecipientAddress",
"payment": {
"amount": "10000000000000000",
"unit": "wei"
}
}
],
"message": "Payment for services"
}
```


}

Success Response

{

"requestId": "a1b2c3d4-e5f6-7890-1234-567890abcdef",

"dltData": [

{

"accountNumber": null,

"address": "0x...YourSourceAddress",

"amount": "10000000000000000",

"atomicSwap": null,

"blockNumber": 1234567,

"data": null,

"feeLimit": "21000",

"feePrice": "10000000000",

"sequence": 42,

"smartContract": {

"data": "0x...",

"extraFields":

},

"toAddress": "0x...RecipientAddress",

"transactionHash": null,

"transactionType": null,

"unlockTime": 0

}

],

"nativeData":

}

Execute a Signed Transaction on a DLT

Endpoint: *POST /v2/execution/transaction*

Description: *Submits a client-signed transaction to the target DLT.*

Request Body

```
{
"requestId": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
"signed": "0xf86...signedTransactionPayload"
}
```

Success Response

```
{
"location": {
"technology": "Ethereum",
"network": "Goerli Testnet"
},
"type": "VALUE_TRANSFER",
"transactionId": "0xabc...TransactionHash",
"status": {
"value": "PENDING"
}
}
```

4.2 SEARCH API

Search for a Transaction

Endpoint: *POST /v2/autoexecution/search/transaction*

Description: *Retrieves transaction details using its hash/ID.*

Request Body

```
{
"transactionId": "0xabc...TransactionHash",
"location": {
"technology": "Ethereum",
"network": "Goerli Testnet"
}
}
```

Success Response

{

"location": {

"technology": "Ethereum",

"network": "Goerli Testnet"

},

"transaction": {

"dlt": "ethereum",

"transactionId": "0xabc...TransactionHash",

"status": { "value": "SUCCESSFUL" },

"block": {

"blockId": "9876543",

"number": 9876543,

"timestamp": "2023-10-27T10:30:00Z"

}

// ... additional transaction details

}

}

Search for an Address Balance

Endpoint: *POST /v2/autoexecution/search/address/balance/*

Description: *Retrieves the balance for a given address.*

Request Body

{

"location": {

"technology": "Ethereum",

"network": "Goerli Testnet"

}

}

Success Response

{

"location": {

"technology": "Ethereum",

"network": "Goerli Testnet"

},

"balances": [

{

"unit": "wei",

"amount": "5000000000000000000"

}

],

"addressId": "0x...Address"

}

Other Search Endpoints

Endpoint Method Description /v2/autoexecution/search/block/ POST Searches for a block by its ID or number.

Endpoint Method Description /v2/autoexecution/search/address/POST Retrieves the transaction sequence for an address. /v2/autoexecution/search/utxo/ POST Searches for a specific UTXO (e.g., on Bitcoin).

4.3 WEBHOOKS API

Create Webhook for Account Updates

Endpoint: *POST /v2/webhook/subscription*

Description: *Subscribes to receive notifications for transaction updates on specified accounts.*

Request Body

```
{
"type": "ACCOUNT",
"ids": [
"0x...AddressToMonitor"
],
"callbackUrl": "https://yourapi.com/webhook/receiver",
"location": {
"technology": "Ethereum",
"network": "Goerli Testnet"
}
}
```

Success Response

```
{
"subscriptionId": "f0e9d8c7-b6a5-4321-fedc-ba9876543210",
"location": {
"technology": "Ethereum",
"network": "Goerli Testnet"
},
"type": "ACCOUNT"
}
```

Other Webhook Endpoints

Endpoint Method Description /v2/webhook/subscription/GET Retrieves information about a webhook. /v2/webhook/subscriptions GET Retrieves a list of all active webhooks. /v2/webhook/subscription/DELETE Deletes a webhook subscription.

5. Data Models

Overledger uses standardized data models to represent DLT-specific information.

Model Object

Location

Origin

Fields

technology, network

originId

Description

Specifies the target DLT and its network (e.g., mainnet, testnet).

The sender's address or identifier.

Destination destinationId, payment (amount, unit) The recipient's address and the payment details.

Model Object

Fields Description

Transaction transactionIdsstatus, block, timestamp,

A standardized representation of a transaction across different DLTs.

Block

Webhook Payload

blockId, number, timestamp, transactionHashes, size

eventId, timestamp, subscriptionId, location, data (transaction)

A standardized representation of a block across different DLTs.

The data structure sent to your callback URL when a monitored event occurs.

Quant Overledger: Technical Overview

This document provides a comprehensive technical overview of the Quant Overledger platform, derived from its official documentation. It covers the

core architecture, authentication mechanisms, data models, API endpoints, and key concepts necessary for developers to integrate with the platform.

1. Architecture Overview

Quant Overledger is an enterprise-grade platform designed to provide interoperability between different Distributed Ledger Technologies (DLTs). Its

architecture is built to abstract the complexities of individual blockchains, offering a single, unified API for developers.

Core Architectural Pillars:

Multi-Chain Interoperability: *Enables seamless transactions and data exchange across a variety of public and private DLTs without requiring*

direct integration with each one.

Standardized API Layer: *Provides a consistent set of APIs for common blockchain operations, such as transaction creation, smart contract*

execution, and token management.

Scalability & Security: *Designed for high-throughput, enterprise-level use cases with a strong focus on security and secure key management.*

Developer Friendliness: *Simplifies blockchain development by eliminating the need for specialized knowledge of disparate DLT protocols.*

Quant Connect: *A deployment component that facilitates the connection of applications to the Overledger network.*

The typical workflow for interacting with Overledger, especially for state-changing operations, follows a ***Prepare -> Sign -> Execute*** *pattern. This*

ensures that the transaction is correctly formatted by Overledger (Prepare), securely signed by the client (Sign), and then submitted for execution

(Execute).

2. Authentication Methods

Overledger employs a multi-faceted approach to authentication and authorization, depending on the API being used. The primary methods observed

are Bearer Token authentication for general API access and a sophisticated payload encryption mechanism for the sensitive Authorise API.

2.1. Bearer Token (OAuth2 / JWT)

For most standard API endpoints, such as preparing a transaction, Overledger uses an OAuth2 Bearer Token.

***Process:** Clients must first obtain a JSON Web Token (JWT) by authenticating with their credentials.*

***Endpoint:** The token is generated via a dedicated endpoint, likely POST /oauth2-token (Note: This endpoint was inaccessible during the*

data gathering phase).

***Usage:** The obtained token is then passed in the Authorization header for subsequent API calls.*

Authorization: Bearer <YOUR_JWT_TOKEN>

2.2. Authorise API (Payload Encryption)

For highly sensitive operations like user management, key creation, and transaction signing, the Authorise API mandates a robust end-to-end

encryption scheme. This ensures that the payload is unreadable by any intermediary, including Quant.

Encryption Flow:

*1. **Generate Symmetric Key:** Create a random 128-bit AES key.*

*2. **Encrypt Payload:** Encrypt the JSON request body using the AES key.*

*3. **Encrypt Symmetric Key:** Encrypt the AES key from step 1 using a provided RSA public key.*

*4. **Send Request:** Make the API call, passing the encrypted payload and the encrypted AES key in separate HTTP headers.*

Required HTTP Headers:

Header

encryptedPayload

Description

The AES-128 encrypted JSON request body.

encryptedSymmetricKeyThe RSA-encrypted AES key used to encrypt the payload. Authorization The standard Bearer <YOUR_JWT_TOKEN> token.

This dual-layer encryption is used for endpoints related to creating users, creating keys, and getting user details.

3. Core Data Models

The following data models are central to interacting with the Overledger API, particularly for managing QRC20 tokens.

3.1. Location Object

The location object specifies the target DLT for the transaction.

```
{
"technology": "Ethereum",
"network": "Ethereum Goerli Testnet"
}
```

3.2. PrepareMintTransactionRequestSchema

This schema is used to prepare a transaction for minting new QRC20 tokens.

```
{
"location": {
"technology": "string",
"network": "string"
},
"type": "Payment",
"urgency": "Normal",
"requestDetails": {
"tokenName": "string",
"amount": "string",
"recipient": {
"accountId": "string"
}
```

}

}

Field Descriptions:

Field Type Description

Field Type Description

location

type

urgency

object The target DLT. See **Location Object**.

string The type of transaction. For QRC20 mint, use Payment.

string The desired transaction processing speed. Enum: Normal, Fast, Urgent.

requestDetailsobject Contains the specifics of the mint operation.

tokenName

amount

recipient

accountId

string The name of the QRC20 token to mint. string The number of tokens to mint (as a string).

object The account that will receive the newly minted tokens.

string The address of the recipient.

3.3. PrepareBurnTransactionRequestSchema

This schema is used to prepare a transaction for burning (destroying) existing QRC20 tokens.


```
{
"location": {
"technology": "string",
"network": "string"
},
"type": "Payment",
"urgency": "Normal",
"requestDetails": {
"tokenName": "string",
"amount": "string"
}
}
```


Note: The structure is similar to the mint request, but it does not require a recipient as the tokens are being removed from the owner's supply.

3.4. Transaction Preparation Response

A successful response from a preparation request provides the necessary data for the client to sign and execute the transaction.


```
{
"requestId": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
"fee": {
"amount": "10000",
"unit": "gwei"
},
"nativeData": {
```

"smartContract": {

"function": {

"name": "mint",

"input": [

{

"type": "address",

"value": "0x...recipientAddress"

},

{

"type": "uint256",

"value": "500"

}

]

}

},

"transaction": {

"nonce": "10",

"gas": "50000",

"value": "0",

"data": "0x...encodedFunctionCallData",

"chainId": 5

}

}

}

4. API Endpoints

The following API endpoints were identified as key functionalities of the Overledger platform.

Method Endpoint Purpose Authentication

Prepares a transaction to mint or burn QRC20

POST /v2/preparation/transactions/supplytokens. The body must conform to Bearer Token PrepareMint... or PrepareBurn... schemas.

POST

/authorise/create-user (hypothesized)

Creates a new user within the Authorise

system. The request payload must be Payload encrypted. Automatically generates one key for Encryption the new user.

Creates a new cryptographic key for an POST /authorise/create-key (hypothesized) existing user. The request payload must be

encrypted.

Payload Encryption

GET

POST

/authorise/get-user-details (hypothesized)

/oauth2-token (hypothesized)

Retrieves details for a user. The request must use the payload encryption scheme.

Retrieves an OAuth2 bearer token (JWT) for authenticating API requests. *(Note: Endpoint was unavailable during analysis)*

Payload Encryption

Client Credentials

> *5. Supported DLTs*

Overledger supports a wide array of blockchain networks. While a definitive list was not available, the documentation places a clear emphasis on **EVM**

***(Ethereum Virtual Machine) compatible chains**. This includes networks like Ethereum mainnet and its various testnets (e.g., Goerli). The*

location object in API requests confirms that developers must specify both the technology (e.g., "Ethereum") and the network (e.g., "Ethereum

Goerli Testnet").

6. Key Concepts and Terminology

***Quant Smart Tokens:** Tokens created and managed via the Overledger platform. They are available in two main categories:*

***Fungible (QRC20):** Interchangeable tokens compliant with the ERC20 standard, suitable for payments, digital currencies, and other*

value transfers.

***Non-Fungible (QRC721):** Unique tokens compliant with the ERC721 standard, ideal for representing unique assets like digital*

collectibles or property titles.

***Token Tiers (Fungible):** QRC20 tokens are offered in two tiers:*

***Base Tier:** Provides standard token functionalities.*

***Flex Tier:** Offers advanced capabilities, including programmatic **minting** and **burning** of tokens.*

***Minting:** The process of creating new tokens and adding them to the total supply. This function is available for Flex tier QRC20 tokens.*

***Burning:** The process of permanently destroying existing tokens, removing them from circulation. This is also a feature of the Flex tier.*

***Authorise API:** A specialized set of endpoints within Overledger dedicated to secure user management, key management, and transaction*

signing, utilizing a mandatory payload encryption scheme.

***Prepare, Sign, Execute:** The fundamental three-step workflow for executing any state-changing operation on a DLT via Overledger. This*

pattern decouples transaction construction from signing, enhancing security.

Project Charter: Quant Overledger to Coinbase Mesh Connector PoC

1. Background and Rationale

A comprehensive investigation into integrating Quant Overledger with Coinbase Mesh confirmed the technical feasibility of a full-scale connector. The

analysis highlighted a strong architectural alignment, particularly in the Prepare -> Sign -> Execute transaction lifecycle.

However, the investigation also identified significant risks and uncertainties associated with the maturity and operational model of Coinbase Mesh:

Operational Overhead: *The requirement to self-host a full DLT node for each Mesh implementation presents considerable resource and*

maintenance challenges.

Documentation Gaps: *Key areas, particularly around complex smart contract interactions (/construction/payloads), are poorly*

documented, introducing implementation risks.

Ecosystem Maturity: *Coinbase Mesh has limited community adoption and publicly available resources, which could hinder development and*

troubleshooting.

Given these findings, a full-scale development effort is premature. A targeted, limited-scope ***Proof-of-Concept (PoC)*** *is the most prudent path*

forward. This PoC will validate the core translation logic for fundamental operations and provide tangible data on the operational challenges of the

Mesh ecosystem before a larger investment is considered.

2. Proof-of-Concept Objectives

The primary objectives of this PoC are:

1. ***Validate Core Functionality:*** *Prove the technical feasibility of translating fundamental Quant Overledger read (account balance) and write*

(asset transfer) API calls into the corresponding Coinbase Mesh API format.

2. ***De-Risk Key Assumptions:*** *Directly address the major risks identified in the initial investigation by implementing the Prepare -> Execute*

flow and documenting the real-world effort required to set up and manage a Mesh node instance.

3. ***Produce a Functional Artifact:*** *Deliver a working, stateless middleware connector that can serve as a technical foundation and a*

demonstrable asset for stakeholders.

*4. **Formulate a Data-Driven Recommendation:** Conclude the PoC with a clear Go/No-Go recommendation for a full-scale project, supported by*

empirical evidence and a detailed analysis of the challenges encountered.

3. Scope Definition

To maintain focus and ensure rapid delivery, the scope is strictly defined as follows:

In Scope	Out of Scope
Connector Middleware Service: A stateless service that translates	**Production Deployment:** The connector specific API calls. production use.
Single DLT Network: Implementation will target a single EVM-based network (e.g., Ethereum) using a standard Mesh reference implementation.	
API Function 1 (Read): Implement get account balance translation.	
API Functions 2 & 3 (Write): Implement the prepare transfer and execute transfer transaction flow.	
Local Node Deployment: The PoC will involve deploying and connecting to a self-hosted DLT node and Mesh instance.	
External Signing: The PoC assumes transaction signing occurs outside the connector (e.g., via Overwallet), focusing solely on the prepare and execute steps.	
Multi-Network Support: The PoC will not support multiple DLTs or dynamic connector registration.	
Complex Reads: No support for block, transaction, or smart contract data queries.	
Complex Writes: No support for smart contract function calls or NFT operations.	
User Interface (UI): No UI will be developed; interaction will be API-based.	
Managed Signing Services: The connector will not manage private keys or signing processes.	

4. Success Criteria

The PoC will be deemed successful upon meeting the following criteria:

*1. **Successful Read Operation:** An API call to the connector for an account balance is successfully translated, sent to the Mesh API, and returns*

the correct balance from the underlying ledger.

*2. **Successful Write Operation:** A prepared and signed asset transfer transaction is successfully submitted through the connector to the Mesh*

API, resulting in a verifiable on-chain state change.

*3. **Validated Translation Logic:** The connector correctly decomposes Quant Overledger's high-level intent (Payment) into the granular, atomic*

TRANSFER operations required by Coinbase Mesh, as verified through logging and testing.

*4. **Comprehensive Final Report:** A final report is delivered, containing:*

A demonstration of the working PoC.

A detailed summary of challenges, particularly concerning node setup and Mesh API ambiguities.

A final, evidence-based Go/No-Go recommendation for proceeding with a full implementation.

5. Proposed Technical Architecture

*The PoC will be implemented as a **Stateless Middleware Service** positioned between the client application (consuming Quant Overledger's API*

format) and the Coinbase Mesh endpoint.

Conceptual Flow:

*Client Application → Quant-Style API Request → **[PoC Connector]** → Coinbase Mesh API Request → Self-Hosted Mesh Instance → DLT*

Node

Connector Components:

***API Interface:** Listens for incoming HTTP requests formatted like Quant Overledger API calls.*

***Translation Engine:** The core logic that performs the following translation:*

***Read (get balance):** Maps directly to a request for the Mesh /account/balance endpoint.*

> ***Write (prepare transfer):*** *Deconstructs the single Overledger Payment intent into two atomic Mesh TRANSFER operations (a debit*
>
> *and a credit, linked via related_operations) and formats them for the /construction/payloads endpoint.*
>
> ***Mesh Client:*** *A simple client responsible for communicating with the deployed Coinbase Mesh API.*

// Example of the core translation logic to be validated by the PoC

// (Based on initial project findings)

// Deconstruct a single Quant 'Payment' into two atomic Mesh 'TRANSFER' operations

operations.push({

> *operation_identifier: { index: 0 },*
>
> *type: 'TRANSFER',*
>
> *account: ,*
>
> *amount: { value: `-$`, currency: currency }, // Debit from origin*

});

operations.push({

> *operation_identifier: { index: 1 },*
>
> *related_operations: [{ index: 0 }], // Link to the debit operation*
>
> *type: 'TRANSFER',*
>
> *account: ,*
>
> *amount: { value: amount, currency: currency }, // Credit to destination*

});

6. High-Level Task Plan

The project will be executed in four distinct phases:

1. ***Phase 1: Environment Setup & Detailed Design***

Deploy a target DLT node (e.g., Geth).

Deploy and configure the corresponding Coinbase Mesh reference implementation in Docker.

Validate connectivity between the Mesh instance and the DLT node.

Finalize the detailed technical design and API contracts for the connector service.

2. ***Phase 2: Core Implementation***

Develop the translation logic for the get account balance read function.

Develop the translation logic for the prepare transfer and execute transfer write functions.

Implement robust unit tests to verify the correctness of the translation engine.

3. ***Phase 3: Integration & End-to-End Testing***

Integrate the connector service with the live, self-hosted Mesh environment.

Execute end-to-end tests for the full read (balance) and write (transfer) flows.

Log and document all results, issues, and operational friction points encountered.

4. ***Phase 4: Analysis & Reporting***

Analyze PoC results against the defined success criteria.

Prepare the final project report, detailing findings, challenges, and the final recommendation.

Prepare and conduct a live demonstration of the functional PoC for all stakeholders.

Technical Architecture & Design: Quant Overledger to Coinbase Mesh PoC Connector

Document Overview

This document provides the detailed technical architecture, design specifications, and implementation guidelines for the Quant Overledger to

Coinbase Mesh Connector Proof-of-Concept (PoC). It serves as the primary technical reference for the development team. The design prioritizes

validating core functionality and de-risking key assumptions as outlined in the project charter.

1. System Architecture

The connector is designed as a **Stateless Microservice Middleware**. Its sole responsibility is to act as a translation proxy between a client expecting

a Quant Overledger-style API and a self-hosted Coinbase Mesh instance.

1.1. High-Level Architecture Diagram

Client Application: *The system initiating API calls (e.g., Postman, a test script, or an application integrated with Overledger). It is responsible*

for managing state, including the signing of transactions.

PoC Connector: *The stateless middleware service detailed in this document. It performs on-the-fly translation of requests and responses.*

Coinbase Mesh Instance: *A self-hosted instance of the Coinbase Mesh reference implementation, deployed in Docker as per the project*

plan.

DLT Node: *A self-hosted full node for the target EVM network (e.g., Geth for Ethereum), which the Mesh instance communicates with.*

1.2. Component Responsibilities

The PoC Connector is composed of several logical components, each with a distinct responsibility.

Component

API Gateway

Translation Engine

Responsibility

- Listens for incoming HTTP requests on defined endpoints.

- Enforces basic request validation (e.g., correct format, required fields).

- Routes requests to the appropriate internal controller.

- The core logic of the service.

- Converts Quant-style API models to Coinbase Mesh API models.

- Re-formats Mesh responses into the expected Quant-style format.

Implementation Notes

Implemented using the Express.js framework.

A set of pure functions or classes for clear separation of concerns and easy unit testing.

> ***Mesh Interaction Layer***

- An HTTP client responsible for all

communication with the Coinbase Mesh API. Can be implemented using a lightweight HTTP - Manages connection details, request retries client library like axios.

(if necessary), and timeouts.

> *- Provides runtime configuration to other*
>
> ***Configuration*** *components. Loads configuration from environment variables* ***Service*** *- Manages environment-specific variables like for Docker compatibility.*
>
> *the Mesh API endpoint URL.*
>
> ***2. Technology Stack***
>
> *The technology stack is selected for rapid development, ease of deployment, and alignment with modern microservice best practices, suitable for a*
>
> *PoC.*

Category Technology Rationale

Language/Runtime Node.js)(LTS

Asynchronous, event-driven model is ideal for I/O-bound operations like API proxying. Fast development cycle.

Web Framework

Deployment

Database

HTTP Client

Logging

Testing

Express.js

Docker & Docker Compose

None

Axios

Winston

Jest

Minimalist, unopinionated framework that provides robust routing and middleware capabilities without unnecessary overhead.

Ensures a consistent and reproducible environment for the connector, the Mesh instance, and the DLT node. Simplifies setup for developers.

The service is explicitly **stateless**. No database is required. Transaction state is managed by the external client.

Promise-based, mature HTTP client for Node.js, simplifying communication with the Mesh API.

A highly configurable logging library, allowing for structured JSON output which is crucial for analysis.

A popular testing framework for JavaScript, ideal for unit-testing the critical Translation Engine logic.

3. Data Flow

The following sequence diagrams illustrate the end-to-end data flow for the in-scope read and write operations.

3.1. Read Operation: Get Account Balance

This flow demonstrates the translation of a simple read request.

1. ***Client*** *sends a GET request to the PoC Connector's /v2/account//balance endpoint.*

2. ***PoC Connector (API Gateway)*** *receives the request and routes it internally.*

3. ***PoC Connector (Translation Engine)*** *constructs the request body for the Coinbase Mesh /account/balance endpoint, specifying the*

network and account identifier.

4. ***PoC Connector (Mesh Interaction Layer)*** *sends the formatted POST request to the* ***Coinbase Mesh API****.*

5. ***Coinbase Mesh*** *queries the underlying* ***DLT Node*** *for the account balance.*

6. ***DLT Node*** *returns the balance data to* ***Coinbase Mesh****.*

7. ***Coinbase Mesh*** *formats the balance into its standard response structure and sends it back to the* ***PoC Connector****.*

8. ***PoC Connector (Translation Engine)*** *receives the Mesh response and transforms it into the Quant Overledger-style balance format.*

9. ***PoC Connector*** *sends the final translated response back to the* ***Client****.*

3.2. Write Operation: Prepare & Execute Transfer

This flow is a two-part process involving an external signer, which is a key assumption of the PoC.

Part A: Prepare Transaction

1. ***Client*** *sends a POST request to the /v2/preparation/transaction endpoint with a Quant-style payment object.*

2. ***PoC Connector*** *receives the request.*

3. ***PoC Connector (Translation Engine)*** *performs the critical logic: it deconstructs the single Quant Payment intent into an array of two atomic*

Mesh TRANSFER operations (one debit from the origin, one credit to the destination).

4. ***PoC Connector (Mesh Interaction Layer)*** *makes a sequence of calls to the* ***Coinbase Mesh*** */construction endpoints:*

First, /construction/preprocess to validate the operations and get necessary parameters.

Second, /construction/metadata to retrieve network-specific information like nonce and gas estimates.

Third, /construction/payloads with the full set of operations and metadata to get the raw, unsigned transaction data.

5. ***Coinbase Mesh*** *returns the unsigned_transaction string and an array of payloads_to_sign.*

6. ***PoC Connector (Translation Engine)*** *formats this response into the structure expected by a Quant client.*

7. ***PoC Connector*** *sends the unsigned_transaction and signing payloads back to the* ***Client****.*

Part B: Execute Transaction

1. ***Client/External Signer*** *uses the payloads_to_sign to generate cryptographic signatures.*

2. ***Client*** *sends a POST request to the /v2/execution/transaction endpoint, including the original unsigned_transaction and the*

generated signatures.

3. ***PoC Connector*** *receives the request.*

4. ***PoC Connector (Mesh Interaction Layer)*** *makes a sequence of calls to the* ***Coinbase Mesh*** */construction endpoints:*

First, /construction/combine with the unsigned transaction and signatures. ***Coinbase Mesh*** *returns a fully formed, signed*

transaction string.

Second, /construction/submit with the signed transaction string.

5. ***Coinbase Mesh*** *submits the transaction to the* ***DLT Node*** *for inclusion in the network.*

6. ***Coinbase Mesh*** *returns the transaction_identifier (i.e., the transaction hash).*

7. ***PoC Connector (Translation Engine)*** *formats the response.*

8. ***PoC Connector*** *sends the final response containing the transactionId back to the* ***Client****.*

4. API Specification (OpenAPI 3.0)

The following specification defines the API exposed by the PoC Connector. It is designed to mimic the relevant Quant Overledger endpoints for

seamless testing.

openapi: 3.0.1

info:

title: Quant-to-Mesh PoC Connector

description: API for translating Quant Overledger requests to Coinbase Mesh for a PoC.

version: 0.1.0

servers:

- url: http://localhost:8080/v2

paths:

/account//balance:

get:

summary: Get Account Balance

description: Translates a balance request to the Mesh /account/balance endpoint.

parameters:

- name: address

in: path

required: true

schema:

type: string

description: The account address to query.

responses:

'200':

description: Successful balance retrieval.

content:

application/json:

schema:

$ref: '#/components/schemas/BalanceResponse'

'404':

description: Account not found.

'500':

description: Internal server error or error from Mesh API.

/preparation/transaction:

post:

summary: Prepare Transaction

description: Translates a Quant-style payment into an unsigned Mesh transaction.

requestBody:

required: true

content:

application/json:

schema:

$ref: '#/components/schemas/PrepareRequest'

responses:

'200':

description: Successfully prepared transaction.

content:

application/json:

schema:

$ref: '#/components/schemas/PrepareResponse'

'400':

description: Invalid request payload.

'500':

description: Internal server error or error from Mesh API.

/execution/transaction:

post:

summary: Execute Signed Transaction

description: Submits a signed transaction to the Mesh /construction/submit endpoint.

requestBody:

required: true

content:

application/json:

schema:

$ref: '#/components/schemas/ExecuteRequest'

responses:

'200':

description: Transaction successfully submitted.

content:

application/json:

schema:

$ref: '#/components/schemas/ExecuteResponse'

'400':

description: Invalid request payload.

'500':

description: Internal server error or error from Mesh API.

components:

```
schemas:

# REQUESTS

PrepareRequest:

type: object

properties:

location:

type: object

properties:

technology: { type: string, example: 'Ethereum' }

network: { type: string, example: 'Ethereum Goerli Testnet' }

type:

type: string

enum: [Payment]

example: 'Payment'

origin:

type: array

items:

type: object

properties:

address: { type: string, example: '0x...origin_address' }

destination:

type: array

items:

type: object

properties:
```

address: { type: string, example: '0x...destination_address' }

amount: { type: string, example: '10000000000000000' } # 0.01 ETH in Wei

currency: { type: string, example: 'ETH' }

ExecuteRequest:

type: object

properties:

requestId:

dlt: { type: string, example: 'ethereum' }

signed:

type: object # This structure will directly mirror the response from /preparation/transaction

properties:

unsigned_transaction:

signatures:

type: array

items:

type: object

properties:

signing_payload:

public_key:

signature:

RESPONSES

BalanceResponse:

```
type: object

properties:

address:

balances:

type: array

items:

type: object

properties:

currency: { type: string, example: 'ETH' }

value: { type: string, example: '5000000000000000000' } # 5 ETH in Wei
```

PrepareResponse:

```
type: object

properties:

requestId:

dlt: { type: string, example: 'ethereum' }

nativeData: # Pass-through the critical data from Mesh

type: object

properties:

unsigned_transaction:

payloads_to_sign:

type: array

items:
```

ExecuteResponse:

type: object

properties:

type: { type: string, example: 'TRANSACTION' }

dlt: { type: string, example: 'ethereum' }

status: { type: string, example: 'SUCCESSFUL' }

transactionId: { type: string, example: '0x...' }

5. Translation Logic

The core of the connector is the logic that maps between the Quant Overledger and Coinbase Mesh data models.

5.1. Read: get account balance

This is a direct mapping.

Quant-Style Request

GET /v2/account//balance

Path Param: address

Coinbase Mesh Request (/account/balance)

POST /account/balance

Body: network_identifier (from config), account_identifier: { address: address }

Response Mapping: *The balances array from the Mesh response is mapped directly to the balances array in the connector's response.*

5.2. Write: prepare transfer

This translation is more complex and represents the primary challenge to be validated by the PoC.

Core Principle: *A single high-level Quant Payment intent must be deconstructed into a set of granular, atomic operations as required by the*

Coinbase Mesh /construction/payloads endpoint. For a simple transfer, this involves two TRANSFER operations.

Illustrative Logic Snippet: *This code demonstrates the creation of the two-operation list from a single payment request.*

```
// Input: A parsed Quant-style PrepareRequest body

// Output: The 'operations' array for the Mesh /construction/payloads request

const operations = [];

// Operation 1: Debit from the origin account

operations.push({

operation_identifier: { index: 0 },

type: 'TRANSFER',

status: '', // Status is left empty for construction endpoints

account: {

address: request.origin[0].address,

},

amount: {

value: `-${request.destination[0].amount}`, // Amount is negative for debit

currency: {

symbol: request.destination[0].currency,

decimals: 18 // This should be dynamically determined or configured

},
```

```
},

});

// Operation 2: Credit to the destination account

operations.push({

operation_identifier: { index: 1 },

related_operations: [{ index: 0 }], // Crucially links this credit to the debit

type: 'TRANSFER',

status: '',

account: {

address: request.destination[0].address,

},

amount: {

value: request.destination[0].amount, // Amount is positive for credit

currency: {

symbol: request.destination[0].currency,

decimals: 18

},

},

});

// This 'operations' array is then sent to the /construction/payloads endpoint
```

6. Error Handling & Logging

A robust strategy for error handling and logging is essential for debugging and analyzing the PoC's performance.

6.1. Error Handling Strategy

The connector will differentiate between client-side errors, its own internal errors, and downstream errors from the Mesh API. A standardized error

response will be used.

Standard Error Response Body:

{

"error": "ERROR_CODE",

"message": "A descriptive error message.",

"details": {

"originalError": "Optional: error details passed through from Mesh API"

}

}

Error Condition

HTTP Status

error Code Action

Invalid client request (e.g., 400 Bad missing fields) Request

INVALID_REQUEST

Reject the request immediately with a clear message about the validation failure.

Resource not found (e.g., balance for non-existent account)

404 Not Found

NOT_FOUND Pass through the 404 status from the Mesh API.

Error returned from Coinbase Mesh API

502 Bad Gateway

Forward the error message from Mesh in the details DOWNSTREAM_ERRORfield. This indicates the connector is working but the

downstream service has an issue.

Internal connector failure (e.g., config error)

500 Internal Server Error

INTERNAL_ERROR

Return a generic server error message. Log the detailed stack trace internally for debugging.

6.2. Logging Framework

Library: *Winston.js will be used.*

Format: *All logs will be written to stdout as* ***structured JSON****. This is critical for compatibility with container orchestration and log*

aggregation tools.

Log Entry Structure: *Every log entry will contain a minimum set of fields for traceability.*

{

"level": "info",

"timestamp": "2023-10-27T10:00:00.123Z",

"requestId": "a1b2c3d4-e5f6-7890-g1h2-i3j4k5l6m7n8",

"operation": "prepare-transaction",

"message": "Successfully prepared transaction for signing.",

"dlt": "ethereum",

"origin": "0x...origin",

"destination": "0x...destination"

}

Log Levels:

error: Unhandled exceptions, failed API calls to Mesh, and critical failures.

warn: Handled exceptions or unexpected conditions that do not stop the request (e.g., Mesh API taking too long to respond).

info: High-level logging of the request lifecycle (e.g., "Request received", "Translation complete", "Response sent").

debug: Detailed, verbose logging for development, including full (sanitized) request/response payloads. This will be disabled by

default.

Quant-to-Mesh Connector: Technical Architecture & Design

Specification

*This document provides the complete technical design for the **Quant-to-Mesh Connector**, a middleware service that exposes a Quant Overledger-*

compatible API and translates requests to a standard Coinbase Mesh implementation. It serves as the primary engineering blueprint for development.

1. System Architecture

The connector is designed as a modular, stateless service that acts as a translation and orchestration layer. It sits between a client application built for

Quant Overledger and a target blockchain network exposed via a Coinbase Mesh-compliant API.

1.1 High-Level Overview

The architecture decouples request handling, data transformation, and external API communication into distinct logical components. This promotes

maintainability, testability, and scalability.

1.2 Component Breakdown

Component Responsibility Key Functions

Quant API Adapter

Exposes Quant Overledger-compatible API endpoints.

- Receives incoming HTTP requests (/v2/preparation/*, /v2/execution/*).

- Validates request schemas against Overledger specifications.

- Orchestrates the internal workflow by calling other components.

Data

Mapper & *The core translation engine.* ***Transformer***

- Maps Overledger data models to Mesh data models.

- Translates high-level Overledger intents (e.g., mint a tokenName) into low-level Mesh Operation arrays.

- Utilizes a **Token Definition Registry** to resolve tokenName to contract details.

Mesh API Client

State Manager

Manages all communication with the downstream Mesh implementation.

Manages the transient state required for the multi-step transaction flow.

- Constructs and sends requests to Mesh endpoints (/construction/payloads, /construction/combine, /construction/submit).

- Handles HTTP-level concerns like headers, timeouts, and retries.

- Generates a unique requestId for each prepare call. - Caches the mapping between the requestId and the unsigned_transaction received from Mesh.

- Uses a time-to-live (TTL) cache (e.g., Redis, in-memory cache) to store this state.

Token Definition Registry

- Provides the contractAddress, functionSignature, and parameter A configurable lookup service details needed to translate an Overledger tokenName into a Mesh for token information. - Can be implemented as a simple configuration file (YAML/JSON)

or a database.

2. Data Flow

The connector orchestrates the Prepare -> Sign -> Execute workflow by translating calls between the two platforms. The client remains

responsible for signing, ensuring private keys are never exposed to the connector.

End-to-End Transaction Submission Flow

Step-by-Step Breakdown:

1. ***Preparation (Client -> Connector -> Mesh):***

a. The client sends a high-level request (e.g., mint a QRC20 token) to the connector's /v2/preparation/transactions/supply

endpoint.

b. ***Quant API Adapter*** *validates the request and generates a unique internal requestId.*

c. ***Data Transformer*** *parses the Overledger request. It uses the tokenName to look up the contract address and function signature from*

the ***Token Definition Registry****.*

d. The transformer constructs a Mesh Operation array that represents the mint intent.

e. ***Mesh API Client*** *sends this Operation array to the target Mesh implementation's /construction/payloads endpoint.*

f. The Mesh implementation returns an unsigned_transaction and signing_payloads.

g. ***State Manager*** *caches the unsigned_transaction against the requestId.*

h. ***Data Transformer*** *maps the Mesh signing_payloads into the Overledger nativeData format.*

i. The connector returns the requestId and nativeData to the client.

2. ***Signing (Client-Side):***

a. The client uses its own wallet or HSM to sign the nativeData (specifically, the signing payloads contained within it).

3. ***Execution (Client -> Connector -> Mesh):***

a. The client sends the signedTransaction (the signature) and the original requestId to the connector's

/v2/execution/transaction endpoint.

*b. **Quant API Adapter** receives the request.*

*c. **State Manager** uses the requestId to retrieve the cached unsigned_transaction.*

*d. **Mesh API Client** makes two sequential calls to the Mesh implementation:*

i. POST /construction/combine: Sends the unsigned_transaction and the client's signature. Mesh returns a fully formed, network-ready signed_transaction.

ii. POST /construction/submit: Sends the signed_transaction from the combine step to be broadcast to the DLT.

e. The Mesh implementation returns a transaction_identifier (the transaction hash).

f. The connector returns this identifier to the client in the expected Overledger format.

3. Data Model Mapping & Transformation Logic

This section defines the precise rules for translating data structures between Overledger and Mesh.

3.1 Request: Overledger PrepareMintto Mesh Payloads

***Input:** Overledger PrepareMintTransactionRequestSchema*

```
{
"location": { "technology": "Ethereum", "network": "Goerli" },
"type": "Payment",
"requestDetails": {
"tokenName": "MyFlexToken",
"amount": "5000000000000000000",
"recipient": { "accountId": "0xRecipientAddress" }
}
}
```

Transformation Logic:

1. Map location to network_identifier.

*2. Use tokenName ("MyFlexToken") to query the **Token Definition Registry**.*

Registry Entry Example (config.yaml):

tokens:

MyFlexToken:

contractAddress: "0xContractAddress"

decimals: 18

functions:

mint:

signature: "mint(address,uint256)"

param_order: ["to", "amount"]

3. Construct the operations array using the Unified Data Model's CONTRACT_CALL structure as a template.

***Output:** Mesh construction/payloads Request*

{

"network_identifier": { "blockchain": "ethereum", "network": "goerli" },

"operations": [

{

"operation_identifier": { "index": 0 },

"type": "CONTRACT_CALL",

"account": { "address": "0xMinterAddress" }, // The 'from' address, must be supplied or inferred

"amount": { "value": "0", "currency": { "symbol": "ETH", "decimals": 18 } }, // Native currency for gas

"metadata": {

"contractAddress": "0xContractAddress",

"functionName": "mint",

"functionSignature": "mint(address,uint256)",

"parameters": [

{ "name": "to", "type": "address", "value": "0xRecipientAddress" },

{ "name": "amount", "type": "uint256", "value": "5000000000000000000" }

]

}

}

],

"public_keys": [/ ... provided by client ... */]*

}

3.2 Response: Mesh Payloadsto Overledger Preparation Response

Input: *Mesh /construction/payloads Response*

{

```
"unsigned_transaction": "0x...", // Opaque string representing the unsigned tx

"payloads": [

{

"address": "0xMinterAddress",

"hex_bytes": "0x...bytesToSign...",

"signature_type": "ecdsa_recovery"

}

]
```

}

Transformation Logic:

*The unsigned_transaction string is **cached** by the State Manager.*

The payloads array is transformed into Overledger's nativeData structure. The connector must reverse-engineer the transaction details

(nonce, gas, etc.) from the hex_bytes or assume a structure consistent with EVM chains, as Overledger's nativeData is not a generic

standard. A pragmatic approach is to pass the signing payload directly.

Output: *Overledger Preparation Response*

```
{

"requestId": "generated-uuid-by-connector",

"nativeData": {

// Best-effort mapping to Overledger's known structure for client compatibility.

// The critical part is passing the data to be signed.

"transaction": {

"data": "0x...bytesToSign...", // This is the 'hex_bytes' from Mesh

"from": "0xMinterAddress"

},

// The raw signing payload for sophisticated clients

"signingPayloads": [

{

"address": "0xMinterAddress",

"hex_bytes": "0x...bytesToSign...",

"signature_type": "ecdsa_recovery"

}

]

}
```

}

4. API Endpoint Mapping

Overledger API Call

Corresponding Mesh API Sequence

Connector Actions

1. Generate requestId.

2. Translate Overledger intent to Mesh operations. /v2/preparation/transactions/*/construction/payloads3. Call Mesh /payloads.

unsigned_transaction requestId 5. Return transformed response.

(Offline Signing) *(None)*

The client signs the payload received from the preparation step.

1. POST

POST /v2/execution/transaction/construction/combine POST

/construction/submit

1. Retrieve unsigned_transaction from cache using requestId.

2. Call Mesh /combine with unsigned data and client signature.

3. Call Mesh /submit with the result from /combine. 4. Return the transaction_identifier.

Account Balance Query

Block Query

POST /account/balance

POST /block

Direct translation of accountId and location to account_identifier and network_identifier.

Direct translation of block identifier.

> ***5. Error Handling & Resiliency***
>
> *A robust connector must gracefully handle failures at every stage of the process.*

Error Scenario

Detection Strategy & Response

> ***Error Scenario***
>
> ***Invalid Client Request***

Detection

Schema validation fails at the **Quant API Adapter**.

Strategy & Response

Reject with a 400 Bad Request status and a descriptive error message mirroring Overledger's format.

> ***Downstream*** *HTTP connection errors (e.g.,* ***Mesh API*** *timeout, 503 Service Unavailable)* ***Unavailable*** *from the* ***Mesh API Client****.*
>
> ***Invalid Mesh*** *Mesh API returns a 4xx status* ***Request*** *code.*

- Implement an **exponential backoff retry** mechanism (e.g., 3 retries over 15 seconds).

- If all retries fail, return a 503 Service Unavailable to the client. Log the failure critically.

This indicates a bug in the connector's **Data Transformer**. Log the full request and response for debugging. Return a 500 Internal Server Error to the client.

> ***Transaction Failure (Mesh Submit)***

Mesh /submit endpoint returns an error from the DLT (e.g., "insufficient funds", "bad nonce").

Forward the specific DLT error message to the client within a 400 Bad Request or 500 Internal Server Error response, depending on the error type. Log the error.

> *The **State Manager** cannot write to or read from its cache (e.g., Redis is down).*

Return a 500 Internal Server Error. This is a critical failure of the connector's infrastructure.

Logging Standard: All logs should be structured (e.g., JSON) and include a correlationId (can be the requestId) to trace a single transaction's

journey through all components.

6. Security Considerations

Security is paramount. The connector's design adheres to the principle of least privilege and minimizes its attack surface.

Stateless Key Management: The connector MUST NOT handle, store, or have any access to user private keys. *The entire architecture*

is built around the Prepare -> Sign -> Execute workflow, which ensures that signing is a client-side responsibility.

Configuration Security: *All sensitive configuration data, such as API keys for the downstream Mesh implementation or database credentials*

for the Token Registry, must be managed through secure environment variables or a dedicated secrets management service (e.g., HashiCorp

Vault, AWS Secrets Manager). They must not be hardcoded.

Input Validation: *The* ***Quant API Adapter*** *must rigorously validate all incoming data against expected schemas to prevent injection attacks or*

malformed data from propagating to downstream services.

Denial of Service (DoS) Mitigation: *The service should be deployed behind a reverse proxy or API gateway that provides rate limiting and*

request throttling to protect it and the downstream Mesh implementation from abuse.

State Cache Security: *If a shared cache like Redis is used for the* ***State Manager****, it must be secured with network policies and authentication*

to prevent unauthorized access to in-flight transaction data.

7. Technology Stack Recommendations

The choice of technology should prioritize performance, developer productivity, and alignment with the existing blockchain ecosystem.

Category Recommended Justification Alternatives

Programming Language

Node.js (TypeScript): - **Ecosystem Alignment:** The Mesh standard, Excellent for rapid API

reference implementations, and SDK (mesh- development with a vast

sdk-go) are all written in Go. This provides a ecosystem. Type safety from significant advantage for development and TypeScript is a major benefit. community support. **Python:** Strong data

- **Performance:** Go's compiled nature and manipulation libraries, but may strong concurrency model are ideal for building have performance limitations high-throughput network services. under high load compared to

Go.

Web Framework

Lightweight, high-performance frameworks with

minimal overhead, suitable for building REST **Express / Fastify** (for Node.js) APIs.

Category Recommended Justification Alternatives

State Cache

Deployment

Redis

Docker / Kubernetes

- Industry-standard in-memory data store. **In-memory cache** (e.g., Go's - Provides persistence and TTL features, which

are perfect for the **State Manager**.

- Highly scalable and performant. scale horizontally.

- **Containerization:** Ensures a consistent Serverless (e.g., AWS runtime environment and simplifies deployment. Lambda): Could be viable but - **Scalability:** Kubernetes provides auto- may introduce complexity with scaling, self-healing, and robust management managing connection state to for a production-grade service. the Mesh API.

Project Foundation and Scoping Document

1. Technology Analysis

This section provides a technical assessment of two key interoperability platforms, Coinbase Mesh and Quant Overledger, and a comparative analysis

to identify synergies and points of friction.

1.1. Coinbase Mesh: Technical Overview

Coinbase Mesh is an open-source, blockchain-agnostic specification designed to standardize interactions with DLTs through a universal API layer. It

aims to simplify and improve the reliability of blockchain integrations.

***Core Architecture** Mesh operates as a middleware layer between a client application and a blockchain node. The standard deployment model*

*mandates packaging a **Mesh Implementation** (the API server) and its corresponding **Blockchain Node** together in a single Docker container.*

***API Specification:** Defined in OpenAPI 3.0, it specifies a RESTful interface with standardized data models.*

***Mesh Implementation:** A service that translates standardized Mesh API calls into the native RPC calls of a specific blockchain. Coinbase*

provides reference implementations for Bitcoin and Ethereum in Golang.

***Developer Tooling:** Includes mesh-cli for compliance testing and mesh-sdk-go to accelerate development.*

***Data Models and API Specification** The API is divided into two primary services:*

*1. **Data API (Read Operations):** Used for querying blockchain state.*

Endpoints: /network/, /account/balance, /block, /block/transaction, /mempool, /call.*

Key Models: AccountIdentifier, Amount, Block, Transaction. Chain-specific details can be included in a generic metadata

field.

*2. **Construction API (Write Operations):** Follows a secure, stateless flow to create and broadcast transactions.*

***Flow:** /construction/payloads (prepare) → sign (offline) → /construction/combine (attach signature) →*

/construction/submit (broadcast).

***Offline Capability:** All Construction API endpoints, except /construction/submit, can be called without a connection to a node,*

enabling secure offline transaction creation.

Key Limitations and Risks

Operational Overhead: *The requirement to run a full blockchain node co-located with each Mesh implementation presents significant*

resource, cost, and maintenance challenges.

Documentation Gaps: *The specification lacks clarity for complex operations, particularly for preparing smart contract calls*

(/construction/payloads) and read-only calls (/call).

Limited Ecosystem: *The standard has not achieved widespread adoption, resulting in scarce community support and a development*

ecosystem heavily centered on Golang.

Contradictory Information: *The official documentation mandating a local node appears to be contradicted by an environment variable*

(GethEnv) in the reference Ethereum implementation that allows connecting to an external node. This ambiguity is a critical concern.

1.2. Quant Network Overledger V3: Technical Overview

Quant Overledger is an enterprise-grade blockchain interoperability platform that functions as a universal, blockchain-agnostic API gateway,

abstracting DLT complexities through a single interface.

Core Architecture *Overledger is an orchestration layer that operates on top of existing blockchains without requiring protocol modifications.*

Layered Model:

Transaction Layer: *Interfaces with DLTs.*

Messaging Layer: *Abstracts and standardizes data from ledgers.*

Filtering & Ordering Layer: *Validates and orders messages for cross-chain logic.*

Application Layer: *Manages application-specific messages.*

API-First Model: *The primary interface is a REST API gateway built on OpenAPI 3.0, authenticated via OAuth 2.0 (clientId,*

clientSecret).

Event-Driven: *Emphasizes webhooks for real-time, event-driven notifications, a shift from the polling model of V2. This promotes a more*

efficient and scalable architecture.

Data Models and API Specification *The API focuses on a two-part flow: preparing an action and monitoring its outcome via webhooks.*

Key Endpoints:

POST /api/preparations/transactions/smart-contracts/write: Prepares a transaction to execute a smart contract write

function.

POST /api/webhooks/smart-contract-events: Creates a webhook to monitor for specific smart contract events.

POST /api/webhooks/accounts: Creates a webhook to monitor account activity.

Data Models:

The API uses structured models for consistency. Smart contract function parameters are defined by their Solidity types (uint, string,

address, arrays, etc.).

Webhook payloads provide detailed, structured JSON data for on-chain events.

Sample Smart Contract Event Webhook Payload:

{

"type": "smartContractEvent",

"webhookId": "a1b2c3d4-...",

"location": {

"technology": "Ethereum",

"network": "Ethereum Goerli Testnet"

},

"smartContractEventUpdateDetails": {

"smartContractId": "0x...",

"nativeData": {

"transactionHash": "0x...",

"blockHash": "0x...",

"blockNumber": 1234567,

"address": "0x...",

"data": "0x...",

"topics": ["0x..."]

}

}

}

1.3. Comparative Analysis: Mesh vs. Overledger

Feature Coinbase Mesh Quant Overledger (V3 API) Synergy & Friction Analysis

Architecture located DLT node for each API gateway as an orchestration implementation.

Friction: Mesh's node requirement introduces significant operational overhead not present in Overledger's gateway model.

> ***Synergy:*** *Both platforms use a Prepare → Sign → Combine Prepare → Sign → Execute. similar and secure Prepare-*
>
> *→ Submit. RESTful, RESTful preparation, with execution*
>
> *stateless API calls. monitoring via webhooks. approach is more modern and scalable for monitoring.*

Smart Contracts

Interaction via generic /call (read) and /construction/payloads (write) endpoints. **Poorly documented and complex.**

Friction: Mesh's approach to Interaction via specific smart contracts is a major /preparations/transactions/smart-implementation hurdle due to its contracts/write endpoint. Clearer, vagueness. Overledger's API is purpose-built model. more explicit and developer-

> *friendly.*
>
> *Standardized, explicit models for primitives*

Data Models (Account, Block, Transaction). Extensible via metadata fields.

> ***Synergy:*** *Mesh's explicit primitive models are a strong*

Standardized but higher-level foundation. Overledger's event models. Rich, event-driven payloads data models are excellent for via webhooks. application-level integration. A

> *unified model could combine both.*

Ecosystem & Maturity

Open-source, but with a small ecosystem, limited documentation, and a strong bias towards Golang.

Proprietary enterprise platform with a focus on low-code integration (Make, Zapier) and Real-World Assets (RWAs).

Friction: Mesh's lack of adoption and support resources is a significant risk. Overledger is a mature, supported platform but less open for community extension.

2. Standardization Pathway Analysis

This section outlines the path toward establishing a formal international standard for DLT interoperability based on the findings from the technology

analysis.

2.1. ISO/TC 307 and DLT Standards Landscape

***Mandate: ISO/TC 307** is the dedicated ISO technical committee for standardizing blockchain and DLTs. Established in 2016, its scope covers*

terminology, interoperability, governance, security, and use cases.

***Key Standards:** The committee has produced foundational standards essential for creating a common language and framework:*

***ISO 22739:** Defines fundamental **vocabulary** for blockchain and DLTs.*

***ISO 23257:** Provides a **reference architecture**, detailing roles, activities, and components in DLT systems.*

***Relevance:** Any new proposal must align with these existing standards to ensure consistency and leverage the foundational work already*

completed by the committee.

2.2. Process for Submitting a New ISO Standard

The submission of a new standard follows a formal, multi-stage process managed by ISO.

*1. **Proposal Stage:***

*A **New Work Item Proposal (NWIP)** is drafted using the official **ISO Form 4**.*

*The proposal must include a clear **scope, justification of market need, and a nominated project leader**.*

The NWIP is submitted by a national standards body to the relevant technical committee (in this case, ISO/TC 307).

*2. **Committee Approval:***

The proposal is circulated among ISO/TC 307 member countries for a vote.

Approval requires a consensus (typically 2/3 majority) and commitment from at least five member bodies to actively participate in the

work.

*3. **Development Stages:***

Once approved, a working group is formed to develop the standard.

The standard progresses through several draft stages: Working Draft (WD), Committee Draft (CD), Draft International Standard (DIS),

and Final Draft International Standard (FDIS).

Each stage involves review, comments, and voting by committee members.

4. ***Publication:***

After all comments are resolved and a final ballot is passed, the standard is formally published by ISO. The typical timeline from

proposal to publication is around ***3 years****.*

2.3. ISO Formatting and Documentation Guidelines

Templates: *ISO provides official* ***Microsoft Word templates*** *that must be used for drafting standards. These templates include predefined*

styles and structures that align with ISO's editorial rules.

Directives:

ISO/IEC Directives, Part 1: *Outlines the procedures for standards development.*

ISO/IEC Directives, Part 2: *Specifies the rules for the structure and drafting of documents, including clause structure, terminology, and*

referencing.

Content: *Submissions must adhere to these directives for layout, use of normative references, and definitions to ensure clarity and*

consistency.

2.4. Conceptual Framework for a Unified Data Model (For ISO Submission)

Based on the analysis of Mesh and Overledger, a new conceptual framework can be proposed. This model aims to combine the strengths of both

platforms while aligning with the principles of ISO/TC 307.

Core Principles:

Layered Abstraction: *Adopt Overledger's powerful layered architecture to separate concerns.*

Standardized Primitives: *Incorporate Mesh's well-defined, low-level data primitives for universal representation of core DLT concepts.*

> ***Event-Driven Design:*** *Prioritize an event-driven (webhook) model for state change notifications, reflecting modern best practices.*
>
> ***ISO Alignment:*** *Explicitly reference and conform to the vocabulary in* ***ISO 22739*** *and the architectural roles in* ***ISO 23257****.*
>
> ***Proposed Conceptual Layers:***

Layer

1. Protocol Connector Layer

Function

Handles protocol-specific communication with DLT nodes (e.g., JSON-RPC for EVM, Bitcoin P2P).

Inspired By / Aligns With

Conceptually similar to Mesh's implementation logic and Overledger's RCGs.

2. Data Translates native DLT data into a standardized format using **Abstraction** explicit, universal models for AccountIdentifier, Block, **Layer** Transaction, Amount, Currency, and Operation.

3. Function Defines standardized, high-level functions (transfer, **Abstraction** executeContract, readState) and their corresponding payload **Layer** structures for both preparation and event monitoring.

Directly inspired by Coinbase Mesh's explicit data models.

Combines Overledger's API-first approach with Mesh's Construction API flow.

4. Manages authentication, authorization, and policy enforcement for *Aligns with the cross-cutting* accessing the interoperability gateway. Defines standard roles *concerns defined in ISO*

Layer (User, Auditor, Administrator). *23257.*

This framework provides a robust foundation for a new interoperability standard that is both technically sound and strategically aligned with

international standardization efforts.

3. Project Scope & Next Steps

3.1. Project Scope

The primary objective of this project is to ***develop a formal specification for a unified DLT interoperability data model and API****. This specification*

will be informed by the analysis of existing technologies (Coinbase Mesh, Quant Overledger) and will be designed for potential submission as a new

international standard to ***ISO/TC 307****.*

The project encompasses:

In-depth technical analysis of relevant industry solutions.

Synthesis of findings into a coherent conceptual framework.

Formalization of the framework into a detailed specification document.

Adherence to ISO procedural and formatting requirements for a new standard proposal.

3.2. Preparatory Work Completed

This document represents the completion of the initial preparatory phase. Key activities completed include:

A comprehensive technical review and synthesis of Coinbase Mesh.

A technical review of the Quant Network Overledger V3 API.

A comparative analysis identifying technological synergies and friction points.

A detailed summary of the ISO/TC 307 landscape and the formal process for submitting a new standard.

The creation of a high-level conceptual framework for a unified data model.

3.3. Dependencies for Next Phase

The next phase of the project, which involves detailing the ***Function Abstraction Layer*** *of the proposed framework, is critically dependent on*

obtaining more granular technical information.

> ***Critical Dependency:*** *Access to the complete technical documentation for the* ***Quant Overledger Fusion API****. The existing V3*
>
> *documentation provides a high-level view, but the Fusion API documentation is required to understand the low-level data structures and*
>
> *interaction patterns necessary for creating a comprehensive and robust specification.*
>
> ***ISO/WD 17749: Unified Data Model for Blockchain and Distributed Ledger Technology Interaction***
>
> ***Status:*** *Working Draft (WD)*
>
> ***Date:*** *[Current Date]*
>
> ***Submitted by:*** *Expert Working Group on DLT Interoperability*
>
> ***0. Foreword***
>
> *The proliferation of Distributed Ledger Technologies (DLT) has created a fragmented digital landscape. Interacting with multiple ledgers requires*

bespoke integrations for each, leading to significant development overhead and systemic risk. Current solutions fall into two categories: managed API

gateways that abstract complexity but create vendor lock-in, and open-source specifications that promote decentralization but often lack high-level

features and clear implementation guidelines.

This document proposes a unified standard for DLT interaction that harmonizes these two approaches. It aims to provide a robust, extensible, and

developer-friendly data model and API specification. By establishing a common language for describing on-chain operations, this standard will foster

true interoperability, reduce integration costs, and accelerate the adoption of DLT applications.

This proposal leverages the strengths of existing models, primarily using the granular Operation-based structure of Coinbase Mesh as its

foundation, while integrating the rich, high-level metadata concepts from platforms like Quant Overledger through a formally defined, extensible

schema.

1. Scope

This standard specifies:

*A **unified data model** for representing core blockchain concepts, including Accounts, Blocks, Transactions, and atomic Operations.*

*A **standardized transaction lifecycle** based on the Prepare -> Sign -> Execute flow to ensure client-side key security.*

*A **logical RESTful API specification** for reading ledger data and constructing/submitting transactions.*

*A **formal schema for metadata** to allow for extensibility and the inclusion of platform-specific or chain-specific attributes in a structured*

manner.

This standard does not specify:

The underlying DLT protocol or consensus mechanism.

The hosting architecture (e.g., managed service vs. self-hosted node).

Specific authentication or authorization mechanisms, which are left to the implementer.

2. Definitions

> *For the purposes of this document, the following terms and definitions apply.*

Term

Operation

Definition

An atomic, indivisible action intended to alter the state of a ledger, such as a value transfer or a smart contract function call. A transaction may contain one or more Operations.

Transaction A signed data package containing one or more Operations, broadcast to a network for inclusion in

Identifier

Metadata

Payload to Sign

A standardized object used to uniquely identify an on-chain entity, such as an Account, Block, or Transaction.

Supplemental, structured data attached to a primary object (e.g., an Operation or Transaction) that provides additional context not part of the core protocol definition.

A data structure representing the cryptographic message that a user must sign with their private key to authorize a transaction.

3. Core Principles

The design of this standard is guided by the following principles:

Client-Side Security: *The transaction lifecycle must separate transaction preparation from signing, ensuring that private keys never leave the*

control of the end-user.

Explicit State Changes: *Transactions are modeled as a collection of Operation objects, making every intended state change explicit and*

auditable.

Structured Extensibility: *The model must be extensible to support future DLTs and custom features without compromising the core standard.*

This is achieved via a formal metadata schema.

Abstraction and Simplicity: *While granular, the model aims to abstract away the most complex, chain-specific details into a consistent*

interface, simplifying development.

4. Data Model Specification

4.1. Core Identifiers

Identifiers are fundamental to locating resources on a ledger.

Identifier

NetworkIdentifier

AccountIdentifier

BlockIdentifier

Structure

```
{ "blockchain": "string", "network": "string" }
```

```
{ "address": "string", "sub_account": { "address": "string" }, "metadata": }
```

```
{ "index": "integer", "hash": "string" }
```

Rationale

Explicitly defines the target ledger (e.g., "Ethereum", "Mainnet"). Essential for routing in a multi-chain environment.

Formally separates the primary address from sub-account identifiers (e.g., memo/destination tags), a common requirement in UTXO and other models.

Allows for block lookup by either height or hash, providing flexibility.

TransactionIdentifier{ "hash": "string" } The unique, on-chain hash of an executed transaction.

4.2. The OperationModel

The Operation is the foundational building block for all state changes. A transaction's intent is described by a list of Operations.

Structure:

```
{
"operation_identifier": {
"index": 0
},
"related_operations": [
{ "index": 1 }
],
"type": "TRANSFER",
"status": "SUCCESS",
"account": { /* AccountIdentifier */ },
"amount": {
"value": "1000000000000000000",
"currency": {
"symbol": "ETH",
"decimals": 18
}
},
"metadata": { /* Metadata Object */ }
}
```

Key Fields:

operation_identifier: Uniquely identifies an Operation within a Transaction.

type: A string enum representing the operation's category. Standard types include:

TRANSFER: A movement of value.

CONTRACT_CALL: Interaction with a smart contract.

FEE: A network fee associated with another operation.

status: The final on-chain status of the operation (e.g., SUCCESS, FAILURE). This is populated on read, not during construction.

account: The AccountIdentifier affected by the operation.

amount: The value being moved, including currency details. For non-value operations, this can be omitted.

Rationale: *By using a list of Operations, this model can precisely describe complex transactions (e.g., a single transaction that performs a token*

approval and a subsequent transfer) in a standardized way. This is a direct adoption of the Coinbase Mesh philosophy for its clarity and granularity.

4.3. The TransactionModel

A Transaction wraps a list of Operations and includes transaction-wide metadata.

Structure:

```
{

"transaction_identifier": { /* TransactionIdentifier */ },

"operations": [

{ /* Operation Object */ },

{ /* Operation Object */ }

],

"metadata": { /* Metadata Object */ }

}
```

4.4. The Formal Metadata Schema

To incorporate the rich contextual data from platforms like Overledger without creating ambiguity, the metadata object is not a free-form blob. It

follows a defined structure.

Structure:

{

"schema_version": "1.0",

"chain_specific": {

"ethereum_v1": {

"gas_limit": "21000",

"priority_fee": "1000000000"

}

},

"platform_extensions": {

"overledger_v1": {

"request_id": "a1b2c3d4-...",

"urgency": "NORMAL",

"callback_url": "https://myapp.com/webhook"

}

},

"application_specific": {

"invoice_id": "INV-2023-10-27-001"

}

}

Rationale:

Namespacing: *Separating metadata into chain_specific, platform_extensions, and application_specific namespaces*

prevents key collisions and provides clear context.

Versioning: *The schema_version and versioned keys (e.g., ethereum_v1) ensure that changes to the metadata structure are non-*

breaking.

> ***Bridging the Gap:*** *This structure formally allows an implementation to support high-level, platform-specific features (like Overledger's*
>
> *urgency or callback_url) within the open standard, providing the best of both worlds.*
>
> ***5. API Specification: Transaction Lifecycle***
>
> *The API facilitates the secure, stateless Prepare -> Sign -> Execute flow.*

(Note: The diagram illustrates the architectural principle of a client interacting with a compliant endpoint, whether that endpoint is a managed gateway

or a self-hosted implementation.)

Step 1: Prepare Transaction (/construction/payloads)

The client specifies its intent by providing a list of Operations. The server returns the precise data that needs to be signed.

Endpoint: *POST /construction/payloads*

Request Body:

```
{
"network_identifier": { "blockchain": "Ethereum", "network": "Mainnet" },
"operations": [ /* List of intended Operation objects */ ],
"metadata": { /* Optional metadata to influence construction */ }
}
```

Success Response:

```
{
"unsigned_transaction": "0x...", /* Hex-encoded, unsigned transaction */
"payloads_to_sign": [
{
"signing_payload": {
"address": "0x...",
"hex_bytes": "0x...",
"signature_type": "ECDSA_RECOVERABLE"
}
}
]
}
```

Step 2: Sign Transaction (Client-Side)

The client signs the hex_bytes from the payloads_to_sign response using the appropriate private key. ***This step occurs entirely offline and***

outside the scope of the API.

Step 3: Execute Transaction (/construction/submit)

The client submits the fully signed transaction for broadcast to the network.

Endpoint: *POST /construction/submit*

Request Body:

```
{
"network_identifier": { "blockchain": "Ethereum", "network": "Mainnet" },
"signed_transaction": "0x..." /* Hex-encoded, signed transaction */
}
```

Success Response:

```
{
"transaction_identifier": {
"hash": "0x..."
}
}
```

6. Conclusion

This proposed standard offers a pragmatic and powerful path toward genuine DLT interoperability. By combining the granular, chain-agnostic

Operation model with a formal, extensible metadata schema, it accommodates both simple, standardized interactions and complex, platform-

specific features. Adopting this hybrid model will provide a stable foundation for the next generation of cross-chain applications, benefiting developers,

enterprises, and end-users alike.

Proposal for a Unified Distributed Ledger Technology Interaction Data model

Introduction: The Interoperability Challenge

The proliferation of Distributed Ledger Technologies (DLTs) has created a fragmented digital landscape where assets and data are siloed within

disparate ecosystems. Achieving seamless interoperability—the ability for different DLTs to communicate and transact with one another—is paramount

for the maturation of the blockchain industry. Currently, developers face a stark choice between two primary philosophies for DLT interaction:

*1. **The Open Specification Approach (e.g., Coinbase Mesh):** This model promotes a decentralized, open-source standard for interacting with*

DLT nodes. Its strengths lie in providing granular control, infrastructure ownership, and well-defined data primitives for core concepts like

accounts and transactions. However, this approach is hindered by significant operational overhead (requiring co-located full nodes), critical

documentation gaps regarding complex operations like smart contract calls, and limited ecosystem adoption.

*2. **The Abstraction Gateway Approach (e.g., Quant Overledger):** This model offers a centralized, managed Platform-as-a-Service (PaaS) that*

abstracts away DLT complexities behind a single, high-level API. Its key strengths are a streamlined developer experience, a modern event-

driven architecture using webhooks, and explicit support for complex assets like NFTs. Its primary weakness is its proprietary, "black box"

nature, where chain-specific data (nativeData) is opaque, and developers have less control and transparency over the underlying

transaction construction.

This proposal introduces a new, hybrid data model that synthesizes the strengths of both approaches. It aims to provide the low-level clarity and

standardization of Mesh's data primitives while incorporating the superior, event-driven architecture and developer-friendly abstractions of Overledger.

The resulting model is designed to serve as a robust foundation for a formal international standard, simplifying development, enhancing security, and

fostering true DLT interoperability.

1. Scope

This document specifies a unified data model and interaction pattern for communicating with diverse Distributed Ledger Technologies. The scope of

this standard includes:

Standardized Data Structures: *Defining a common, unambiguous representation for core DLT concepts, including networks, accounts,*

blocks, transactions, assets, and operations.

Read Operations: *A model for querying on-chain data, such as account balances, block details, and transaction statuses.*

Write Operations: *A secure, stateless, and offline-capable workflow for constructing, signing, and broadcasting transactions, including simple*

value transfers and complex smart contract interactions.

Event Monitoring: *A standardized, event-driven mechanism for notifying client applications of on-chain events in real-time.*

This standard does not scope key management, identity solutions, or the specific implementation of the underlying protocol connectors. It focuses

exclusively on the interface layer between a client application and an interoperability gateway or node middleware.

2. Normative References

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document.

ISO 22739:2020*, Blockchain and distributed ledger technologies — Vocabulary*

ISO 23257:2022*, Blockchain and distributed ledger technologies — Reference architecture*

3. Terms and Definitions

For the purposes of this document, the terms and definitions given in ISO 22739 and the following apply.

Operation: *The smallest, indivisible component of a transaction that describes a state change, such as a balance change or a smart contract*

function call. A single transaction may contain one or more operations.

Payload: *Data intended for cryptographic signing by a key holder. A secure interaction flow ensures this data is generated in a trusted*

environment and signed offline.

Construction: *The process of creating a valid, network-ready transaction from a set of desired operations.*

Webhook: *An automated, server-to-server notification (HTTP callback) triggered by a predefined on-chain event.*

4. Unified Interaction Model Specification

> *The proposed model is founded on principles of explicit data representation, secure workflow design, and modern, event-driven communication.*
>
> ***4.1 Core Data Primitives***
>
> *These objects form the foundational vocabulary for all interactions. They are heavily inspired by the clarity of the Coinbase Mesh specification.*

Object Fields Description

NetworkIdentifier

AccountIdentifier

technology Uniquely identifies a specific DLT and its network (e.g., technology: "Ethereum", network: "Mainnet"). Combines the clarity of Overledger's

(string) location and Mesh's NetworkIdentifier.

address

(string) Represents a unique account on a DLT. subAccount can hold memo fields (object, (e.g., for Cosmos) or other identifiers.

optional)

Currency

Amount

symbol (string) decimals (integer) metadata (object, optional)

value (string) currency (Currency)

Defines a fungible or non-fungible asset. The metadata object is **required** for tokens and must contain the contractAddress.

Represents a quantity of a specific asset. value is a string to handle arbitrary-precision numbers.

TransactionIdentifier(string) A unique, network-native identifier for a submitted transaction.

4.2 The OperationObject: The Heart of a Transaction

The Operation is the most critical abstraction, transforming implicit transaction effects into an explicit, auditable list. A transaction is simply a

collection of one or more Operation objects.

Field Type Description

operationIdentifierobject An index to uniquely identify the operation within a transaction.

relatedOperations array

type string

status string

An optional array of related operation indices (e.g., for linking a debit and a credit).

The nature of the operation. Standard types include: TRANSFER, FEE, CONTRACT_CALL, MINT, BURN.

The outcome of the operation (e.g., "Success", "Failed"). Left empty during construction.

account AccountIdentifierThe account affected by the operation.

amount Amount

metadata object

The amount of currency being moved. Negative for debits, positive for credits.

Type-specific data. For CONTRACT_CALL, this object contains structured call data.

Example CONTRACT_CALL Metadata:

"metadata": {

"functionName": "mint",

"functionSignature": "mint(address,uint256)",

"parameters": [

{

"name": "to",

"type": "address",

"value": "0x..."

},

{

"name": "amount",

"type": "uint256",

"value": "1000000000000000000"

}

]

}

4.3 Secure Transaction Lifecycle

The model mandates a stateless, four-step construction flow that maximizes security by enabling offline signing. This refines the Coinbase Mesh

workflow.

1. ***Prepare (/construction/payloads):*** *The client specifies its intent by providing a list of Operation objects. The service returns the*

transaction data to be signed (payloads).

2. ***Sign (Offline):*** *The client uses its private key—which never leaves its possession—to sign the payloads.*

3. ***Combine (/construction/combine):*** *The client sends the unsigned transaction data and the generated signatures to the service, which*

assembles the final, network-ready transaction.

4. ***Submit (/construction/submit):*** *The client sends the fully combined and signed transaction to the service for broadcasting to the target*

DLT.

4.4 Event-Driven Monitoring via Webhooks

To provide real-time updates without inefficient polling, the model adopts Overledger's event-driven paradigm.

1. Webhook Registration (POST /webhooks/events)

A client subscribes to events by providing a callback URL and the event details to monitor.

Request Body:

{

```
"type": "ACCOUNT_TRANSACTION",

"location": {

"technology": "Ethereum",

"network": "Goerli"

},

"accountId": "0x...",

"callbackUrl": "https://myapp.com/webhook-receiver"

}
```

2. Webhook Payload Structure

When a monitored event occurs, the service sends a structured POST request to the callbackUrl.

Example Payload:


```
{
"webhookId": "wh-a1b2-c3d4",
"eventId": "evt-e5f6-g7h8",
"timestamp": "2023-10-27T10:00:00Z",
"type": "ACCOUNT_TRANSACTION",
"location": {
"technology": "Ethereum",
"network": "Goerli"
},
"eventDetails": {
"transactionIdentifier": {
"hash": "0x..."
},
"blockIdentifier": {
"hash": "0x...",
"index": 1234567
},
"operations": [
{
"operationIdentifier": { "index": 0 },
"type": "TRANSFER",
"status": "Success",
```

"account": { "address": "0xSENDER..." },

"amount": { "value": "-1000", "currency": { "symbol": "ETH", "decimals": 18 } }

},

{

"operationIdentifier": { "index": 1 },

"type": "TRANSFER",

"status": "Success",

"account": { "address": "0xRECEIVER..." },

"amount": { "value": "1000", "currency": { "symbol": "ETH", "decimals": 18 } }

}

]

}

}

5. Design Justification

This hybrid model is intentionally designed to address the identified shortcomings of existing solutions and establish a superior standard.

Design Choice

Explicit Operation Model

Structured Smart Contract Metadata

Justification

Provides an unambiguous, auditable breakdown of all transaction effects. Eliminates the need for developers to parse opaque, chain-specific data payloads to understand what a transaction did.

Standardizes the way smart contract calls are prepared, defining a clear schema for function names, signatures, and parameters. This greatly simplifies one of the most complex aspects of DLT development.

Addresses Weakness In

Quant Overledger: Replaces the "black box" nativeData with a transparent, standardized structure.

Coinbase Mesh: Solves the problem of the poorly documented and overly generic /call and /construction/payloads endpoints.

Design Choice

Justification Addresses Weakness In

> *Promotes a scalable, real-time, and resource-efficient architecture for monitoring on-chain activity. This is superior to legacy polling methods, which are slow and create unnecessary network load.*
>
> *Enforces a secure-by-design workflow where private keys are never exposed to the interoperability service. The explicit combine step adds transparency to the transaction construction process.*

Creates a universal vocabulary (AccountIdentifier, **Standardized** Currency, etc.) that reduces ambiguity and makes it easier **Primitives** to write code that works across multiple DLTs without

> *modification.*

Coinbase Mesh: Integrates a first-class eventing system, a feature largely absent from the core Mesh specification.

Both: Formalizes and refines the best security practices seen across both platforms into a clear, mandatory lifecycle.

Both: Synthesizes the cleanest concepts from each platform into a single, coherent set of primitives.